Laboratório 8: Q-learning (CliffWalking)

Importações

In [ ]:
```
!pip install gymnasium[toy-text]
!pip install tqdm pandas seaborn
!pip install gdown
```

```
Requirement already satisfied: gymnasium[toy-text] in /usr/local/lib/python3.
dist-packages (1.4.0)
Requirement already satisfied: numpy>=1.22.0 in /usr/local/lib/python3.13/dis
packages (from gymnasium[toy-text]) (2.4.6)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.1
dist-packages (from gymnasium[toy-text]) (3.1.2)
Requirement already satisfied: typing-extensions>=4.12.0 in /usr/local/lib/
python3.13/dist-packages (from gymnasium[toy-text]) (4.16.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/
python3.13/dist-packages (from gymnasium[toy-text]) (0.0.6)
Requirement already satisfied: pygame-ce>=2.1.3 in /usr/local/lib/python3.13/
packages (from gymnasium[toy-text]) (2.5.8)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-package
(4.67.3)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packa
(2.2.3)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-pack
(0.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dis
packages (from pandas) (2.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/pytho
dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist
packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/di
packages (from pandas) (2026.6)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/
python3.13/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.68.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/d
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-pa
(from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-pac
(from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: gdown in /usr/local/lib/python3.13/dist-packag
(5.2.2)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.13/di
packages (from gdown) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-pac
(from gdown) (4.2.7)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.13/d
packages (from gdown) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-package
(from gdown) (4.67.3)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.13/dis
packages (from beautifulsoup4->gdown) (4.0)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/
python3.13/dist-packages (from beautifulsoup4->gdown) (4.16.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/
python3.13/dist-packages (from requests[socks]->gdown) (3.4.9)
```

```
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist
packages (from requests[socks]->gdown) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.1
dist-packages (from requests[socks]->gdown) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.1
dist-packages (from requests[socks]->gdown) (2026.10.23)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/pytho
dist-packages (from requests[socks]->gdown) (1.7.1)
```

In [ ]:
```python
import os
import gdown
from IPython.display import SVG, display
import gymnasium as gym
from gymnasium.wrappers import RecordEpisodeStatistics
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
from typing import NamedTuple, Optional, List, Tuple
from collections import deque
```

Funções auxiliares para visualização

In [ ]:
```python
RL_UTILS_URL = "https://drive.google.com/file/d/1AEJC-YWgeZCr8pZEOOiAg6PN

if not os.path.exists("rl_utils.py"):
    try:
        gdown.download(RL_UTILS_URL, "rl_utils.py", quiet=True, fuzzy=Tru
    except Exception as error:
        print(f"Falha no download: {error}")

from rl_utils import (
    plot_results, plot_comparison,
    simulate_trajectory, plot_trajectory,
    show_one_episode,
    plot_tabular, plot_policy_distributions,
)
```

In [ ]:
```python
FIGURE_URLS = {
    "mapa_cliffwalking":
        "https://drive.google.com/file/d/1x1Poz-iZ3IcW-aLyYCwDMN7DPHlo5_S
    "env_step":
        "https://drive.google.com/file/d/1csEbTeLIz7a8-FDz0xwrd_M_T-_Z0Dj
    "alvos_sarsa_qlearning":
        "https://drive.google.com/file/d/1vV29K0aDUBIeldPAOJ6mRJ7d67Uuh7_
    "epsilon_gulosa":
        "https://drive.google.com/file/d/1LtJYLSN-zlitE6BMPHNTVoIuZcGcOym
    "qlearning_passo":
        "https://drive.google.com/file/d/17R5xHWCJXk7oUfvzicc9lAZ32CMNM90
}
FIGURE_DIR = "figuras"

os.makedirs(FIGURE_DIR, exist_ok=True)
for name, url in FIGURE_URLS.items():
    path = os.path.join(FIGURE_DIR, f"{name}.svg")
    if not os.path.exists(path):
        try:
            gdown.download(url, path, quiet=True, fuzzy=True)
```

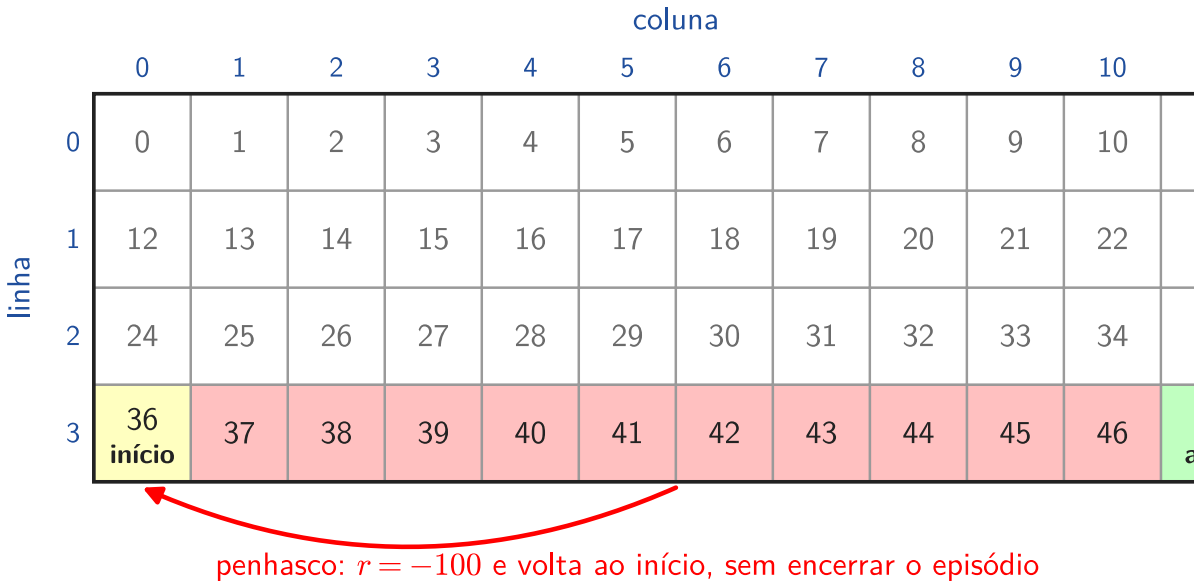
```
        except Exception as error:
            print(f"Falha no download de {name}: {error}")


def show_figure(name: str) -> None:
    path = os.path.join(FIGURE_DIR, f"{name}.svg")
    try:
        display(SVG(filename=path))
    except Exception:
        print(f"Figura indisponível: {name}. Confira o link em FIGURE_URL
```

Ambiente: nova instância

O laboratório usa o `CliffWalking-v1`, do Gymnasium.

In [ ]:
```
show_figure("mapa_cliffwalking")
```



- **Estado:** o índice da célula, `12 * linha + coluna`, que é também a observação devol ambiente.
- **Ações:** quatro; não há a ação de permanecer no lugar.
- **Recompensa:** -1 por passo. Quanto mais curto o caminho, maior a recompensa total, e caminho mais curto passa rente ao penhasco.
- **Penhasco:** entrar nele custa -100 e devolve o agente a $s_0 = 36$, **sem** encerrar o epis
- **Fim do episódio:** há um estado terminal, o alvo. Como um agente que ainda não aprendeu vagar por muito tempo, o episódio também termina ao atingir um limite de passos.
- **Modelo:** não é usado. O agente aprende só com as amostras de experiência devolvidas po `env.step`, e não com um modelo como o `env.P` do FrozenLake (Laboratório 3).

A célula abaixo define as constantes e `make_env`, que cria o ambiente com o limite de passos envolve com o wrapper `RecordEpisodeStatistics`. Cada treinamento deve usar um ambi novo, criado por `make_env`.

Detalhes: `RecordEpisodeStatistics`

- No último passo de cada episódio, acrescenta a `info` a entrada `info["episode"]`, co
  - `"r"`: a recompensa total, isto é, a soma das recompensas, sem desconto;
  - `"l"`: a duração, em passos;
  - `"t"`: o tempo de execução, em segundos.
- Guarda esses valores dos últimos `STATS_BUFFER_LENGTH` episódios nas filas `return_length_queue` e `time_queue`, usadas nas médias da barra de progresso.
- Um ambiente novo começa com as filas vazias.

In [ ]:
```
ENV_ID = "CliffWalking-v1"
MAX_EPISODE_STEPS = 500
STATS_BUFFER_LENGTH = 100
SEED = 42


def make_env(render_mode: Optional[str] = "rgb_array") -> gym.Env:
    env = gym.make(ENV_ID, render_mode=render_mode,
                   max_episode_steps=MAX_EPISODE_STEPS)
    return RecordEpisodeStatistics(env, buffer_length=STATS_BUFFER_LENGTH


env = make_env()
```

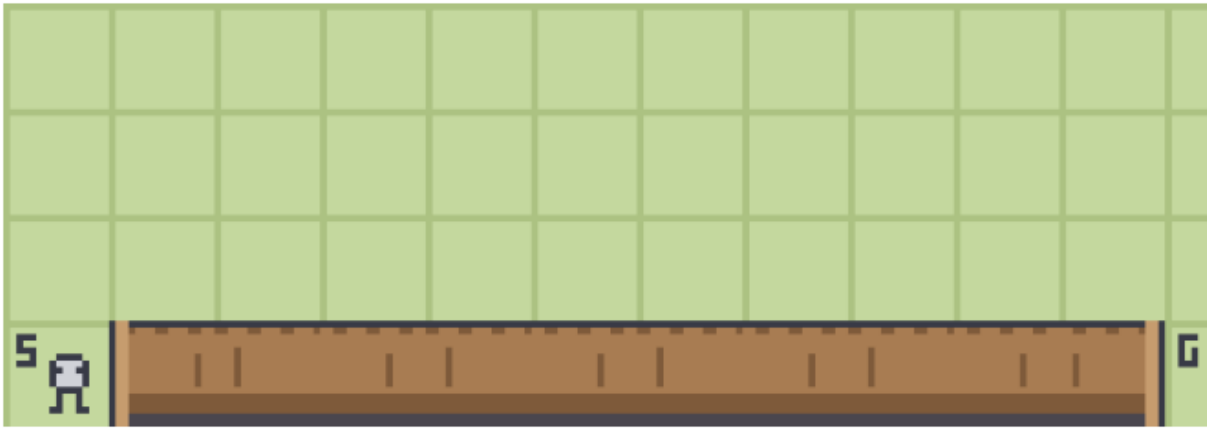
`env.reset` inicia um episódio e devolve o estado inicial s_0 e o dicionário `info`.

In [ ]:
```
obs, info = env.reset(seed=SEED)
obs, info
```

Out[ ]:
```
(36, {'prob': 1})
```

`env.render()` devolve a imagem do estado atual, um array com shape `(altura, largur`

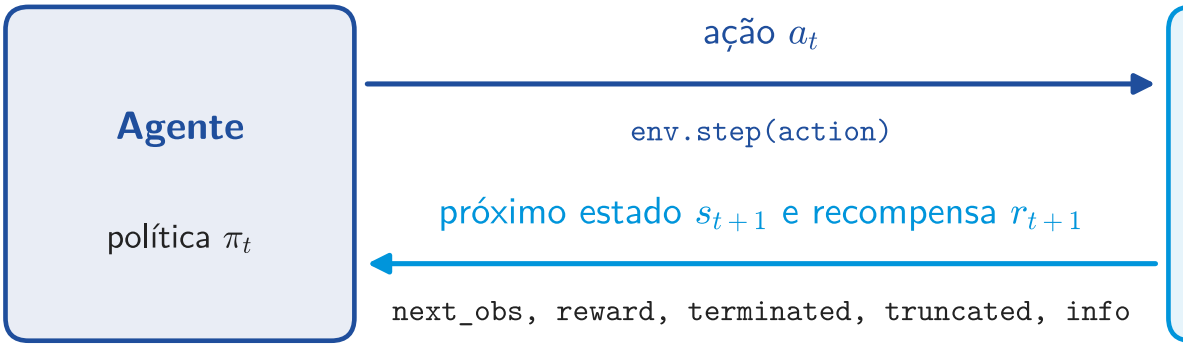
In [ ]:
```
img = env.render()
plt.figure(figsize=(9, 3))
plt.imshow(img)
plt.axis("off")
plt.show()
img.shape
```



Out[ ]:
```
(256, 768, 3)
```

`env.step` executa um passo. Ele recebe só a ação a_t, porque o ambiente já guarda o est
s_t:

In [ ]:
```
show_figure("env_step")
```



Saída	Nomenclatura	Significado
`next_obs`	s_{t+1}	próximo estado
`reward`	r_{t+1}	recompensa do passo
`terminated`	s_{t+1} terminal	`True` ao chegar ao alvo: o episódio acaba, e $v(s_{t+1})=0$
`truncated`	—	`True` ao atingir `MAX_EPISODE_STEPS`: o episódio para, mas s_{t+1} **não** é terminal
`info`	—	dados auxiliares (`"prob"` e, no último passo, `"episode"`)

> **Atenção:** a_t não vem do ambiente: o agente a sorteia em s_t, com a política $\pi_
> antes de chamar `env.step`.

A célula abaixo executa a ação `1` (direita) a partir de $s_0 = 36$, que leva ao penhasco. Obse
`reward`, `next_obs` e `terminated`.

In [ ]:
```
next_obs, reward, terminated, truncated, info = env.step(1)
next_obs, reward, terminated, truncated, info
```

Out[ ]:
```
(36, -100, False, False, {'prob': 1.0})
```

A célula abaixo executa um episódio com ações sorteadas uniformemente por
`env.action_space.sample()`. Agindo ao acaso, o agente dificilmente alcança o alvo, e o
costuma acabar por truncamento; no último passo, `info["episode"]` traz as estatísticas do

In [ ]:
```
env.action_space.seed(SEED)
obs, info = env.reset(seed=SEED)
episode_over = False
while not episode_over:
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)
    episode_over = terminated or truncated
terminated, truncated, info["episode"]
```

Out[ ]:
```
(False, True, {'r': -5945.0, 'l': 500, 't': 0.019501})
```

Algoritmo: Q-learning

O Q-learning escolhe as ações com uma política ϵ-gulosa e corrige a estimativa do pa
(s_t, a_t) **a cada passo**, como o Sarsa do Laboratório 7, mas com outro alvo TD: no lugar do
ação seguinte, entra o maior valor de ação em s_{t+1}. Com isso, q estima diretamente os
de ação ótimos:

$$ \underbrace{q_{t+1}(s_t,a_t)}_{\substack{\text{nova}\\ \text{estimativa}}} \;=\; \underbrace{q_t(
_{\substack{\text{estimativa}\\ \text{atual}}} \;-\; \underbrace{\alpha}_{\substack{\text{taxa de}\\
\text{aprendizado}}} \, \underbrace{\Big[\, q_t(s_t,a_t) \;-\; {\color{red}{\overbrace{r_{t+1} + \gamm
\max_{a \in \mathcal{A}(s_{t+1})} q_t(s_{t+1},a)}^{\text{alvo TD: } \bar q_t}}} \,\Big]}_{\text{erro TD
\delta_t} $$

- $q_t(s,a)$ é a estimativa do valor de ação ótimo de (s,a) no passo t, e só o par visitad
  $q_{t+1}(s,a) = q_t(s,a)$ para todo $(s,a) \neq (s_t,a_t)$.
- Se s_{t+1} é terminal, o futuro vale zero e o alvo é só $\bar q_t = r_{t+1}$.
- Cada atualização usa só $(s_t, a_t, r_{t+1}, s_{t+1})$: a próxima ação não entra no alvo.

Detalhes: a equação que o Q-learning resolve
O alvo $\bar q_t$ é uma amostra do lado direito da equação de otimalidade de Bellman, escrita
valores de ação:

$$q(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a} q(S_{t+1},a) \,\Big|\, S_t = s, A_t = a\Big]

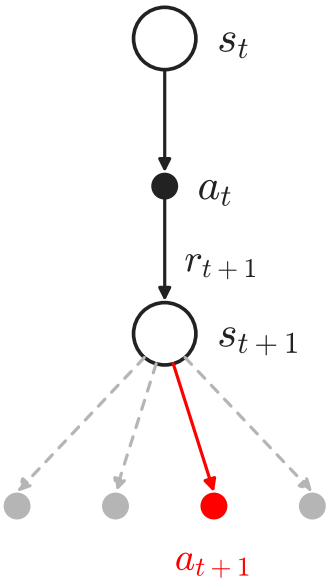
A solução dessa equação são os valores de ação ótimos. O Sarsa resolve a equação de Bellma
política seguida, $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma\, q_\pi(S_{t+1},A_{t+1}) \mid S_t =
a]$, e por isso estima os valores dessa política.

A diferença para o Sarsa está no termo de bootstrap do alvo:

In [ ]:
```
show_figure("alvos_sarsa_qlearning")
```

Sarsa

$$\bar{q}_t = r_{t+1} + \gamma\, q_t(s_{t+1}, a_{t+1})$$



usa só a ação a_{t+1}, sorteada pela política em s_{t+1}: às vezes, uma ação exploratória

Q

$$\bar{q}_t = r_{t+1}$$

usa a maio
seja qual for

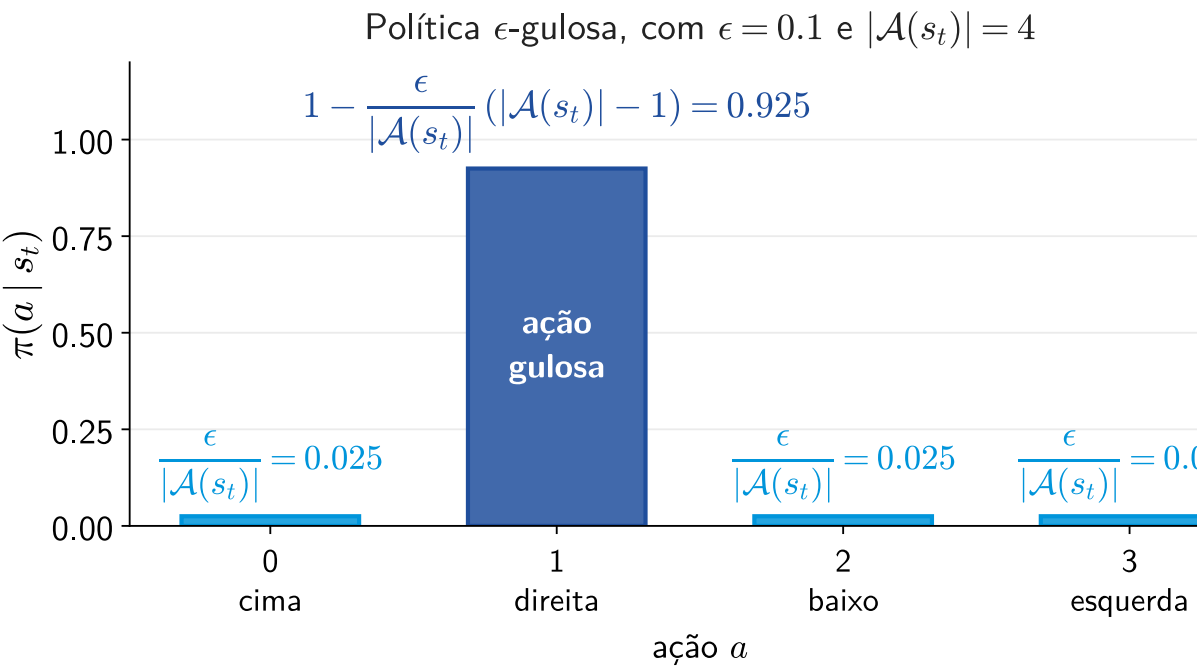
Cada iteração tem dois passos:

1. **Valor de ação:** atualiza $q(s_t, a_t)$ pela equação acima.
2. **Política:** atualiza a política em s_t, que passa a ser ϵ-gulosa em relação às esti atualizadas:

$$ \pi_{t+1}(a \mid s_t) = \begin{cases} 1 - \dfrac{\epsilon}{|\mathcal{A}(s_t)|}\big(|\mathcal{A}(s_ 1\big), & \text{se } a = \arg\max\limits_{a} q_{t+1}(s_t,a) \\[10pt] \dfrac{\epsilon}{|\mathcal{A}(s_t) \text{caso contrário} \end{cases} $$

Com $\epsilon = 0.1$ e as quatro ações do CliffWalking:

In [ ]:
```
show_figure("epsilon_gulosa")
```



> **Atenção:** em relação ao Sarsa, só o alvo muda. As ações continuam sendo escolhidas pela política ϵ-gulosa, mas o alvo usa a estimativa da melhor ação em $s_{t+1} seja qual for a ação que o agente escolher depois.

Implementação. Preencha os blocos marcados:

Parte	Descrição	A implementar
`Params`	reúne os hiperparâmetros	—
`QLearningAgent`	guarda q, π e as contagens de visitas; sorteia as ações; executa os dois passos	`update_policy`, `get_target_value` e `update_Q`
`run_episode_and_train`	executa um episódio, atualizando o agente a cada passo	o laço
`train_agent`	repete os episódios e registra as métricas	—

Hiperparâmetros

`Params` reúne os hiperparâmetros; `n_states` e `n_actions` vêm do ambiente.

Campo	Hiperparâmetro
`n_episodes`	número de episódios, N
`learning_rate`	taxa de aprendizado, α
`discount_factor`	fator de desconto, γ
`epsilon`	parâmetro ϵ da política

Detalhes: como variar um hiperparâmetro com `_replace`

Uma `NamedTuple` não pode ser alterada depois de criada, mas `params._replace(learning_rate=0.5)` devolve uma cópia de `params` com α t
Isso facilita variar um hiperparâmetro de cada vez, como nos itens 1 a 5 da tarefa.

In [ ]:
```python
class Params(NamedTuple):
    n_states: int
    n_actions: int
    n_episodes: int
    learning_rate: float
    discount_factor: float
    epsilon: float
```

Agente

Implemente `update_policy`, `get_target_value` e `update_Q`; os demais métodos já e
prontos. O agente guarda:

Atributo	Conteúdo
`self.Q`	estimativas $q_t(s,a)$ dos valores de ação ótimos, inicializada com zeros
`self.Pi`	política $\pi_t(a \mid s)$, inicializada uniformement `self.Pi[s]` é o vetor de probabilidades das aç `s`
`self.visit_counts`	número de visitas a cada par (s,a)
`self.learning_rate`, `self.discount_factor`, `self.epsilon`	α, γ, ϵ
`self.rng`	gerador de números aleatórios do agente

As três matrizes têm shape `(n_states, n_actions)` e são indexadas por `[estado, aç`
Nos métodos, os argumentos correspondem a:

`obs`	`action`	`reward`	`next_obs`	`terminated`
s_t	a_t	r_{t+1}	s_{t+1}	s_{t+1} é terminal?

Detalhes: métodos prontos

- `get_greedy_action(obs)`: devolve $\arg\max_a q_t(s,a)$; em caso de empate, a açã menor índice.
- `sample_action(obs)`: sorteia a_t com as probabilidades de `self.Pi[obs]`, usan `self.rng`, e devolve `(action, is_exploratory)`, em que `is_exploratory` é ação sorteada difere da gulosa e `0` caso contrário.
- `update(...)`: chama `update_Q` e `update_policy` e registra a visita ao par $(s_t,
- `policy(obs, deterministic)`: devolve a ação gulosa ou sorteia uma ação $\epsilon gulosa; é usado nas visualizações.

A implementar:

- `update_policy(obs)` $\to$ não devolve nada; atualiza a linha `self.Pi[obs]`, o pas pela fórmula ϵ-gulosa da seção **Algoritmo**:
  - a ação gulosa recebe $1 - \dfrac{\epsilon}{|\mathcal{A}(s_t)|}\big(|\mathcal{A}(s_t)| - 1\b
  - cada uma das demais ações recebe $\dfrac{\epsilon}{|\mathcal{A}(s_t)|}$, com $|\math (s_t)|$ = `self.n_actions`.
- `get_target_value(next_obs, reward, terminated)` $\to$ o alvo $\bar q_t$, um real:
  - $\bar q_t = r_{t+1} + \gamma \max_a q_t(s_{t+1},a)$, ou só r_{t+1} se s_{t+1} é te
  - a linha `self.Q[next_obs]` traz as estimativas das quatro ações em s_{t+1}.
- `update_Q(obs, action, next_obs, reward, terminated)` $\to$ o erro TD $\de um número real (escalar):
  - calcula o alvo com `get_target_value` e o erro TD: $\delta_t = q_t(s_t,a_t) - \bar q_
  - atualiza o par visitado: $q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha\,\delta_t$.

> **Atenção:**
>
> - Só `terminated` zera o futuro. No truncamento, s_{t+1} não é terminal e $ \gamma \max_a q_t(s_{t+1},a)$ continua no alvo; por isso os métodos não recebem `truncated`.
> - Em `update_policy`, obtenha a ação gulosa com `self.get_greedy_action(obs)`: é a mesma que `sample_action` usa para marcar as ações exploratórias.

In [ ]:

```
class QLearningAgent:
    def __init__(self, params: Params, seed: int = SEED):
        # Número total de estados do ambiente (48 estados: grade 4x12)
        self.n_states = params.n_states
        # Número de ações possíveis em cada estado (4 ações: 0=Cima, 1=Di
        self.n_actions = params.n_actions
        # Taxa de aprendizado alpha (ex: 0.1): fração de ajuste em direçã
        self.learning_rate = params.learning_rate
        # Fator de desconto gamma (ex: 0.9): ponderação do valor de recom
        self.discount_factor = params.discount_factor
        # Epsilon (ex: 0.1): probabilidade de tomar uma ação aleatória (e
        self.epsilon = params.epsilon
        # Gerador pseudoaleatório com semente fixa para garantir reprodut
        self.rng = np.random.default_rng(seed)
        # Formato da tabela Q: 48 estados x 4 ações
        shape = (self.n_states, self.n_actions)
        # Tabela Q inicializada com zeros: estimativas q(s, a) dos valore
        self.Q = np.zeros(shape)
        # Política comportamental Pi: inicializada uniforme (0.25 para ca
        self.Pi = np.full(shape, 1.0 / self.n_actions)
        # Contador de visitas a cada par (estado, ação)
        self.visit_counts = np.zeros(shape)


    def get_greedy_action(self, obs: int) -> int:
        # Retorna a melhor ação atual no estado obs: argmax_a Q(obs, a)
        # ATENÇÃO: se houver empate em valores de Q, o np.argmax retorna
        return int(np.argmax(self.Q[obs]))
```

```python
    def update_policy(self, obs: int) -> None:
        # Passo 1: identifica qual é a ação de maior valor Q no estado ob
        greedy_action = self.get_greedy_action(obs)
        # Passo 2: probabilidade base de exploração para cada ação (epsil
        base = self.epsilon / self.n_actions
        # Passo 3: probabilidade da ação gulosa: (1 - epsilon) + fatia de
        calc = (1.0 - self.epsilon) + base

        # Passo 4: preenche todas as ações com a probabilidade base de ex
        self.Pi[obs] = base
        # Passo 5: atribui a probabilidade majoritária à melhor ação
        self.Pi[obs, greedy_action] = calc


    def sample_action(self, obs: int) -> Tuple[int, int]:
        # Sorteia a_t segundo a distribuição comportamental epsilon-gulos
        action = int(self.rng.choice(self.n_actions, p=self.Pi[obs]))
        # Flag booleana: indica se a ação sorteada foi uma exploração (di
        is_exploratory = int(action != self.get_greedy_action(obs))
        return action, is_exploratory


    def policy(self, obs: int, deterministic: bool = False) -> int:
        # Se deterministic=True (modo deploy/avaliação pura): retorna a a
        if deterministic:
            return self.get_greedy_action(obs)
        # Se deterministic=False (modo treinamento/comportamental): sorte
        return self.sample_action(obs)[0]


    def get_target_value(self, next_obs: int, reward: float, terminated:
        # Se o próximo estado for terminal (chegou no alvo ou penhasco):
        if terminated:
            target_value = reward
        # DIFERENÇA CRUCIAL DO Q-LEARNING (OFF-POLICY):
        # O alvo usa np.max(self.Q[next_obs]), ou seja, assume que no fut
        # sempre tomará a MELHOR ação possível, independente da ação que
        else:
            target_value = reward + (self.discount_factor * np.max(self.Q
        return target_value


    def update_Q(self, obs: int, action: int, next_obs: int, reward: floa
                 terminated: bool) -> float:
        # 1. Calcula o alvo TD off-policy usando np.max
        target_value = self.get_target_value(next_obs, reward, terminated
        # 2. Calcula o erro TD
        td_error = self.Q[obs, action] - target_value
        # 3. Atualiza Q(s, a) dando um passo proporcional a alpha para re
        self.Q[obs, action] -= self.learning_rate * td_error
        return td_error

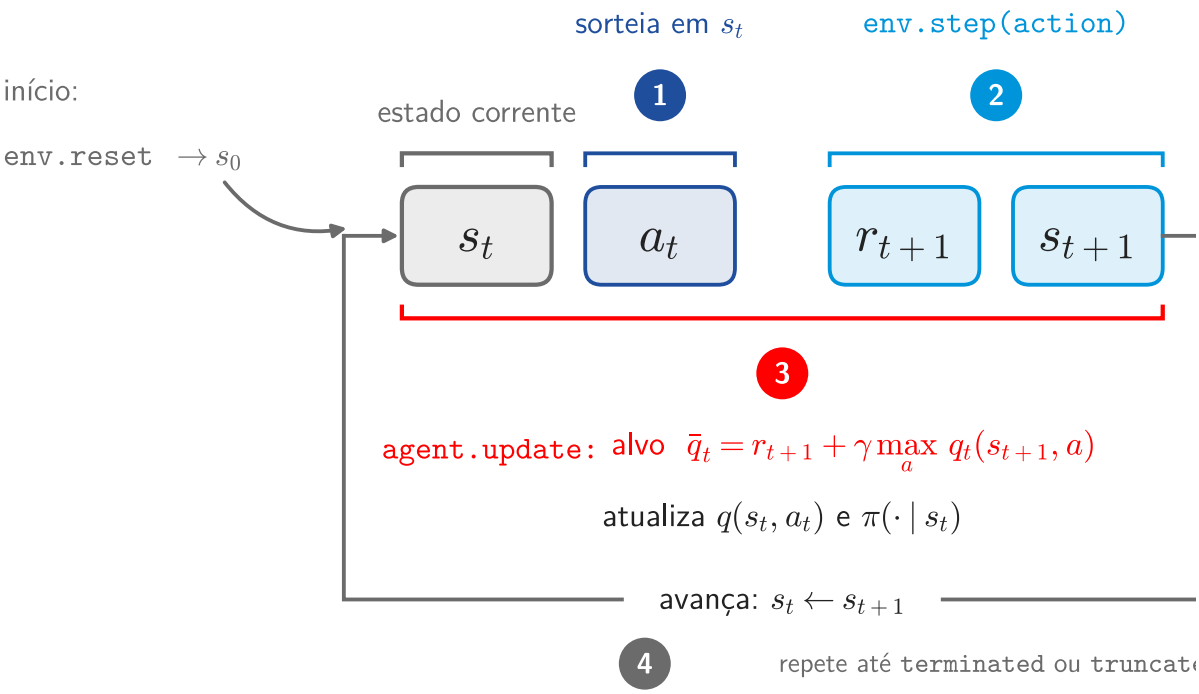

    def update(self, obs: int, action: int, next_obs: int, reward: float,
               terminated: bool) -> None:
        # 1. Atualiza a tabela Q com a transição observada
        self.update_Q(obs, action, next_obs, reward, terminated)
        # 2. Atualiza a política comportamental no estado obs para que re
```

```
        self.update_policy(obs)
        # 3. Incrementa o contador de visitas do par (estado, ação)
        self.visit_counts[obs, action] += 1
```

Episódio de treinamento

Implemente o laço de `run_episode_and_train`. O início já está pronto: `env.reset` devo
s_0. Cada iteração do laço segue os passos da figura:

In [ ]:
```
show_figure("qlearning_passo")
```



1. sorteia a_t em s_t, com a política ϵ-gulosa, chamando `agent.sample_act`
2. executa a_t com `env.step`, que devolve r_{t+1} e s_{t+1};
3. atualiza o agente com $(s_t, a_t, r_{t+1}, s_{t+1})$, chamando `agent.update`;
4. avança: $s_t \leftarrow s_{t+1}$.

O laço termina quando `terminated` ou `truncated` é `True`.

> **Atenção:**
>
> - Não há a_{t+1} no laço: o alvo não a usa, e a ação do passo seguinte é sorteada no início dele, com a política já atualizada.
> - Passe `terminated` ao agente, e **não** `terminated or truncated`.
> - Some em `n_exploratory_actions` o `is_exploratory` de cada ação executada.

Retorno (já pronto):

- `info["episode"]`: recompensa total, duração e tempo do episódio;
- `n_exploratory_actions`: número de ações não gulosas executadas;
- `terminated`: o sinal do último passo.

In [ ]:
```python
def run_episode_and_train(agent: QLearningAgent, env: gym.Env, seed: int
    # Inicia o episódio resetando o ambiente para s_0 (estado 36)
    obs, _info = env.reset(seed=seed)

    episode_over = False
    n_exploratory_actions = 0
    terminated = False
    info = {}

    while not episode_over:
        # 1. Sorteia a ação a_t a partir da política comportamental epsil
        action, is_exploratory = agent.sample_action(obs)
        # Contabiliza estatística de exploração
        n_exploratory_actions += is_exploratory

        # 2. Executa a ação a_t no ambiente e observa o próximo estado ne
        next_obs, reward, terminated, truncated, info = env.step(action)

        # 3. VANTAGEM OFF-POLICY DO Q-LEARNING:
        # Atualiza a tabela Q imediatamente usando next_obs (sem precisar
        # O método update_Q usará internamente np.max(Q[next_obs])
        agent.update(obs, action, next_obs, reward, terminated)

        # 4. Avança o estado atual para o próximo
        obs = next_obs
        # Verifica se encerrou naturalmente (terminated) ou atingiu o lim
        episode_over = terminated or truncated

    return info["episode"], n_exploratory_actions, terminated
```

Laço externo

`train_agent` já está pronta. Ela repete `run_episode_and_train` por `n_episodes` ep
cada um com uma semente sorteada pelo gerador do agente, e devolve as séries numa
`TrainingHistory`:

Campo	Série, por episódio
`returns`	recompensa total
`lengths`	duração, em passos
`times`	tempo de execução, em segundos
`exploration_rates`	proporção de ações não gulosas executadas
`successes`	1 se o episódio terminou no alvo; como o alvo é o único estado terminal do CliffWalking, o sucesso é o próprio `terminated`

A barra de progresso mostra as médias dos últimos `STATS_BUFFER_LENGTH` episódios, calcu
com as filas do `RecordEpisodeStatistics`. O argumento opcional `run_episode` troca
que executa cada episódio; ele só é usado no item 5 da tarefa, para treinar o Sarsa com o laço

Detalhes: a tupla nomeada `TrainingHistory`
Cada campo pode ser acessado pelo nome, como em `history.returns`, e a tupla também
desempacotada na ordem em que os campos foram definidos.

In [ ]:
```
class TrainingHistory(NamedTuple):
    returns: List[float]
    lengths: List[int]
    times: List[float]
    exploration_rates: List[float]
    successes: List[int]


def train_agent(agent, env: gym.Env, n_episodes: int,
                run_episode=None) -> TrainingHistory:
    if run_episode is None:
        run_episode = run_episode_and_train
    returns, lengths, times, exploration_rates, successes = [], [], [], [
    success_queue = deque(maxlen=env.return_queue.maxlen)
    pbar = tqdm(range(n_episodes), desc="Treinando", unit="episódio")

    for episode in pbar:
        seed = int(agent.rng.integers(2**32))
        episode_data, n_exploratory_actions, terminated = run_episode(
            agent=agent, env=env, seed=seed)

        episode_length = int(episode_data["l"])
        exploration_rate = n_exploratory_actions / episode_length
        success = int(terminated)

        returns.append(float(episode_data["r"]))
        lengths.append(episode_length)
        times.append(float(episode_data["t"]))
        exploration_rates.append(exploration_rate)
        successes.append(success)
        success_queue.append(success)

        pbar.set_postfix({
            "recompensa_media": f"{np.mean(env.return_queue):.1f}",
            "passos_medio": f"{np.mean(env.length_queue):.1f}",
            "tempo_medio": f"{np.mean(env.time_queue):.3f}",
            "exploracao": f"{exploration_rate:.3f}",
            "sucesso": f"{np.mean(success_queue):.3f}",
        })

    return TrainingHistory(returns, lengths, times, exploration_rates, su
```

Experimento

Simulação

A célula abaixo treina o Q-learning na configuração base, com um ambiente (`train_env`) e u
novos. O gerador do agente controla todos os sorteios e o CliffWalking é determinístico: com a r
semente, a execução é reprodutível.

In [ ]:
```
params = Params(
    n_states=env.observation_space.n,
    n_actions=env.action_space.n,
    n_episodes=1000,
    learning_rate=0.1,
    discount_factor=0.9,
```

```
    epsilon=0.1,
)

train_env = make_env()
agent = QLearningAgent(params, seed=SEED)
history = train_agent(agent, train_env, params.n_episodes)
```

Como q estima os valores de ação ótimos, os valores de estado saem do valor da melhor açã
cada estado:

$$v(s) = \max_a q(s,a)$$

In [ ]:
```
V = np.max(agent.Q, axis=1)
```

Avaliação da política ϵ-gulosa

O Q-learning estima os valores de ação ótimos, mas, no treinamento, as ações são escolhidas p
política ϵ-gulosa em relação aos valores de ação estimados, guardada em `agent.Pi`
`evaluate_policy`, já pronta, mede o desempenho dessa política, com exploração: executa
`n_episodes` episódios (100, por padrão) num ambiente novo, criado por `make_env`, sortea
ação com as probabilidades de `agent.Pi`, **sem** atualizar q nem π. Ela devolve:

Métrica	Significado
`chegada ao alvo`	proporção dos episódios que terminam no alvo
`recompensa total média`	média da recompensa total dos episódios
`duração média`	média da duração dos episódios, em passos

Os sorteios usam um gerador próprio, criado com `seed`.

> **Atenção:** as curvas de `plot_results` vêm do treinamento, em que a política muda a
> cada atualização, inclusive dentro de um episódio. Elas mostram como o agente
> aprendeu; o desempenho da política ϵ-gulosa ao fim do treinamento é o que
> `evaluate_policy` mede, com q e π fixos.

In [ ]:
```
def evaluate_policy(agent, n_episodes: int = 100, seed: int = SEED) -> pd
    rng = np.random.default_rng(seed)
    eval_env = make_env(render_mode=None)
    returns, lengths, successes = [], [], []

    for _ in range(n_episodes):
        obs, _info = eval_env.reset(seed=int(rng.integers(2**32)))
        terminated = truncated = False
        while not (terminated or truncated):
            action = int(rng.choice(agent.n_actions, p=agent.Pi[obs]))
            obs, _reward, terminated, truncated, info = eval_env.step(act
        returns.append(float(info["episode"]["r"]))
        lengths.append(int(info["episode"]["l"]))
        successes.append(int(terminated))

    return pd.Series({
        "chegada ao alvo": np.mean(successes),
```

```
        "recompensa total média": np.mean(returns),
        "duração média": np.mean(lengths),
    })
```

Na configuração base:

In [ ]:
```
evaluate_policy(agent)
```

Out[ ]:

	0
chegada ao alvo	1.00
recompensa total média	-52.45
duração média	20.77

dtype: float64

Visualização

As figuras vêm das funções do `rl_utils`, importadas no início do notebook:

Função	Descrição
`plot_results`	recompensa total, duração, taxa de exploração e sucesso por ep com a média móvel de `STATS_BUFFER_LENGTH` episódios
`simulate_trajectory`	executa um episódio num ambiente próprio e devolve a trajetória padrão, a trajetória gulosa (definida abaixo)
`plot_trajectory`	o caminho sobre a grade, com o penhasco em vermelho e o alv verde
`show_one_episode`	a animação de um episódio com a política ϵ-gulosa, ex incluída
`plot_tabular`	mapas de calor de $q(s,a)$, $\pi(a \mid s)$ ou $v(s)$, escolhido argumento `kind`
`plot_policy_distributions`	as probabilidades das quatro ações em cada estado
`plot_comparison`	as médias móveis de várias execuções num só gráfico, com lege (usada na tarefa)

> **Atenção:** cada mapa de calor calcula a escala de cores a partir dos próprios valores, e a mesma cor pode representar números diferentes em figuras diferentes. Para comparar figuras, use os números anotados nas células.

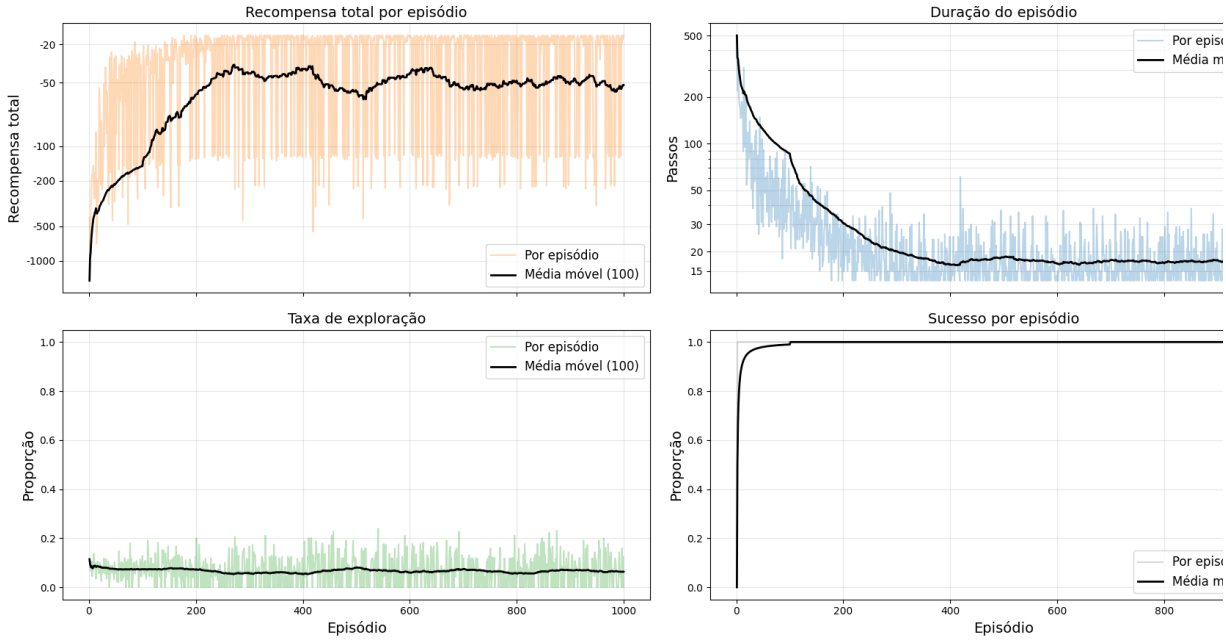
Detalhes: escalas dos eixos e gráficos lado a lado

- Em `plot_results` e `plot_comparison`, o eixo da recompensa total usa a escala `sy` quando há valores abaixo de -100: linear entre -100 e 100 e logarítmica fora dessa f eixo da duração usa a escala logarítmica quando a duração varia muito. Nos demais casos escala é linear.
- `plot_comparison` e `plot_trajectory` aceitam o argumento `ax`, o painel da figura o gráfico é desenhado. Com `ax`, elas não chamam `plt.show()`, e a figura é exibida un só, ao final. Para as curvas de duração e de recompensa total lado a lado:

```
fig, axs = plt.subplots(1, 2, figsize=(18, 5))
plot_comparison(histories, metric="lengths", ax=axs[0])
plot_comparison(histories, metric="returns", ax=axs[1])
plt.tight_layout()
plt.show()
```

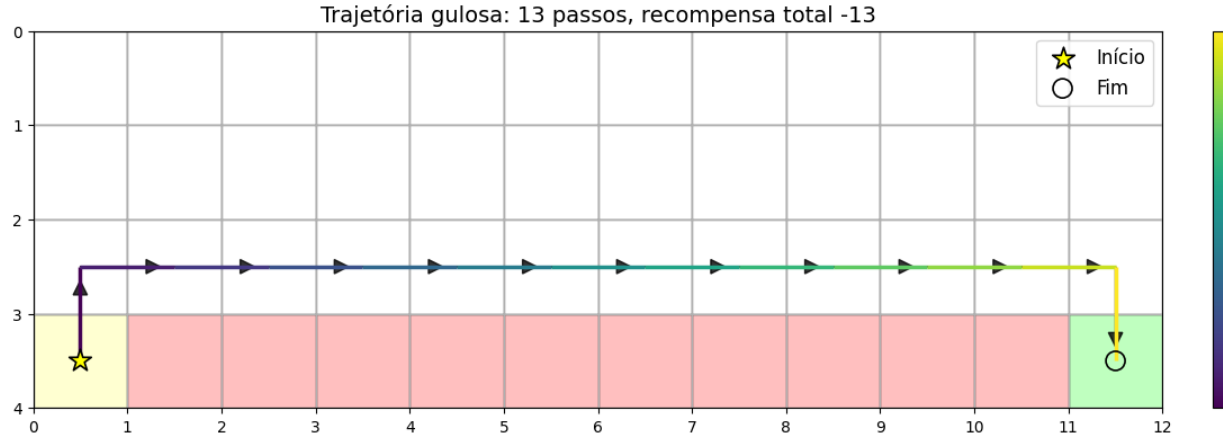
em que `histories` é um dicionário que associa o rótulo de cada configuração (o texto da legenda) ao histórico devolvido por `train_agent`.

In [ ]:
```
df_results = plot_results(
    history.returns, history.lengths, history.exploration_rates,
    history.successes, window=STATS_BUFFER_LENGTH)
```



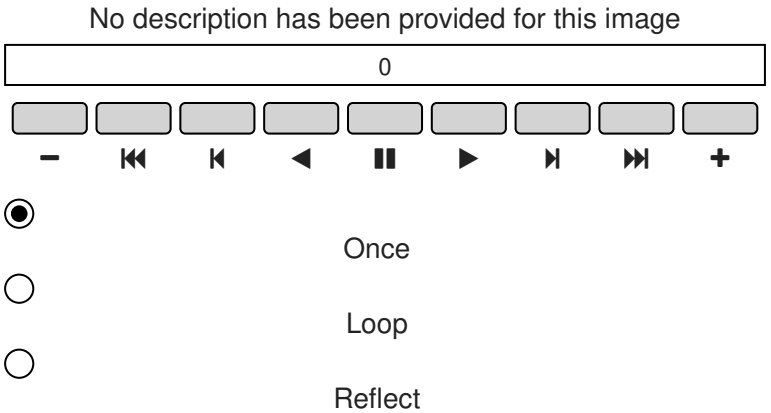
Trajetória gulosa: a trajetória obtida executando, em cada estado, a ação de maior valor estima \arg\max_a q(s,a)$, sem exploração. Ela mostra o caminho indicado pelos valores de ação apre desempenho com exploração é o da política ϵ-gulosa, medido por `evaluate_polic` $\epsilon = 0.1$, a política ϵ-gulosa escolhe a ação gulosa com probabilidade 0.925 er estado.

In [ ]:
```
states, actions, rewards = simulate_trajectory(agent, ENV_ID)
title = (f"Trajetória gulosa: {len(actions)} passos, "
         f"recompensa total {sum(rewards):.0f}")
_ = plot_trajectory(ENV_ID, states, title=title)
```



In [ ]:
```
show_one_episode(agent.policy, ENV_ID, seed=SEED)
```

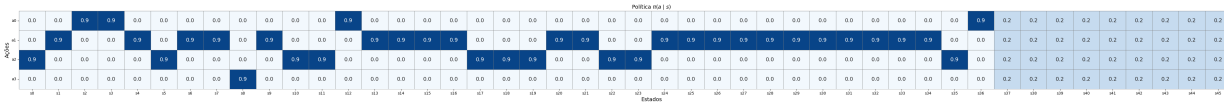
Out[ ]:



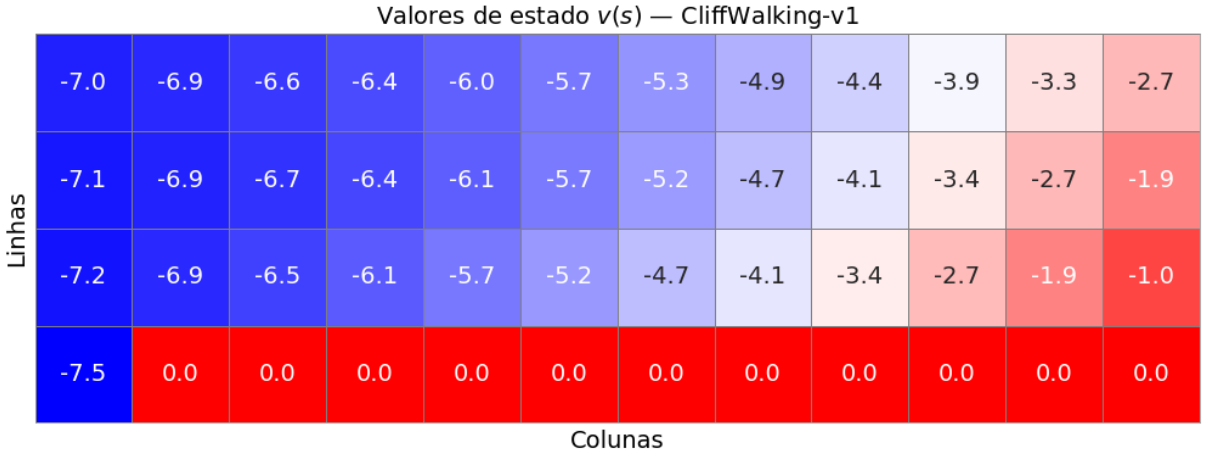
In [ ]:
```
_ = plot_tabular(agent.Q, kind="Q")
```



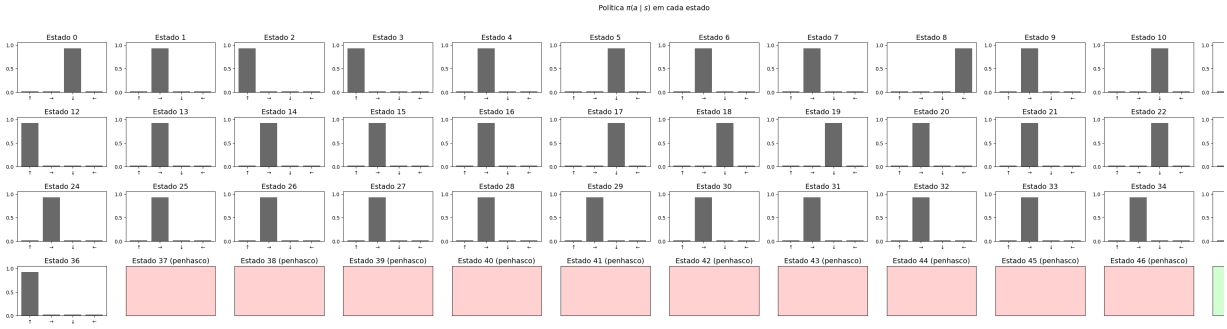
In [ ]:
```
_ = plot_tabular(agent.Pi, kind="Pi")
```



In [ ]:
```
_ = plot_tabular(V, kind="V", env_id=ENV_ID, center_zero=False)
```



In [ ]:
```
plot_policy_distributions(agent.Pi, ENV_ID)
```



Tarefa (em trios):

Responda no próprio notebook. Cada item tem uma seção **Resposta**, com:

- uma célula de código, para os experimentos, as figuras e as tabelas;
- uma célula de texto, com **uma resposta por pergunta**, identificada pelo mesmo rótulo: (a),

Acrescente células se precisar, mas não apague nem reordene os enunciados.

Configuração base (baseline)

Parâmetro	Valor
ambiente	`CliffWalking-v1`, com `MAX_EPISODE_STEPS = 500`
número de episódios, N	`n_episodes = 1000`
taxa de aprendizado, $ \alpha$	`learning_rate = 0.1`
fator de desconto, γ	`discount_factor = 0.9`
parâmetro ϵ	`epsilon = 0.1`
janela das médias móveis	`STATS_BUFFER_LENGTH = 100` episódios
semente	`SEED = 42`, com ambiente (`make_env`) e agente novos em cada execução

São os mesmos valores do Laboratório 7, para que o item 5 possa comparar o Q-learning com o

Mostrar

- **Figuras:**
  - configuração base: as figuras da seção **Visualização** (curvas de `plot_results`, tra gulosa e mapas de calor de $q(s,a)$, $\pi(a \mid s)$ e $v(s)$);
  - itens 1 a 4: as curvas de duração e de recompensa total de todas as execuções do iten sobrepostas e com legenda (`plot_comparison`), e a trajetória gulosa de cada exec
  - item 5: para cada valor de ϵ, as curvas de duração e de recompensa total do $ do Q-learning, sobrepostas e com legenda, e a trajetória gulosa de cada algoritmo.
- **Tabela**, para cada execução dos itens 1 a 5:
  - chegada ao alvo, recompensa total média e duração média da política ϵ-gulos medidas com `evaluate_policy`;
  - passos e recompensa total da trajetória gulosa.
- **Discussão:** uma resposta curta por pergunta, na célula de texto da **Resposta**.

Entregáveis

- Um notebook (`.ipynb`) por laboratório da Unidade 2, entregue ao fim da unidade, com a respondida nas seções **Resposta**. Nesta unidade não há relatório em PDF.
- O notebook deve **executar do início ao fim sem erros** em um kernel novo (*Restart & Run* as saídas exibidas correspondendo a essa execução. Notebooks que não executarem por c serão penalizados.
- A entrega é **única por grupo**, com os integrantes identificados no início do notebook.

🧭 Guia de Estudo e Apresentação (Laboratório 8 - Q-Learning)

🟢 O QUE VOCÊ DEVE DETALHAR E DESTACAR (Fale com confiança):

- **Q-Learning é Off-Policy:** O agente separa duas políticas: a que ele joga para coletar da (comportamental $\epsilonhBcgulosa) e a que ele aprende (política-alvo ótima $\max_a
- **Caminho Ótimo Estrito (13 passos):** A trajetória gulosa do Q-learning é o caminho mai matematicamente possível (13 passos rente ao penhasco, retorno -13). Como o alvo usa \max_a$, ele assume que no futuro sempre acertará, ignorando os tropeços da exploraç
- **Diferença Crucial no Código vs SARSA:** O Q-learning não precisa sortear {t+1}$ antes atualização. Ele atualiza direto com $\max_a Q(s_{t+1}, a)$.
- **O Duelo SARSA vs Q-Learning (Item 5):** O SARSA ganha durante o treinamento (curva recompensa muito melhores, em torno de -20 a -30 vs -50 a -70 do Q-learning). Porém, n política gulosa final (deploy sem exploração), o Q-learning ganha (-13 vs -15 do SARSA)

🔴 O QUE PODE GERAR DÚVIDA / PEGADINHAS DA BANCA (Atenção redobrada):

- **Por que a recompensa média com evaluate_policy no Q-Learning é tão horrível (-5 -70)?** Porque o caminho ótimo que ele aprendeu passa colado ao penhasco! Na hora de com $\epsilon=0.1$, a cada 10 passos ele sorteia uma ação aleatória; como ele está na beirada, qualquer ação para baixo derruba ele no penhasco (-100 de penalidade imediat
- **O Q-Learning é sempre melhor que o SARSA?** NÃO! Depende do contexto. Se você e treinando um robô físico real onde erros custam caro (quebrar o robô), o SARSA é muito seguro (Safe RL). Se for em simulação onde o que importa é a rota perfeita no final, o Q-Learning é ideal.
- **O que acontece quando $\epsilon o 00** Sem exploração aleatória, o risco de cair desap O SARSA perde o 'medo' do abismo e converge exatamente para a mesma rota ótima de passos do Q-Learning.

0. Implementação

🟢 O QUE DETALHAR NA TAREFA 0:

- **(a) Trajetória gulosa de 13 passos:** O agente aprende o caminho mais curto geométrico passos, retorno -13) colado à borda do penhasco. Isso acontece pela natureza *off-policy*: operador $\max_a Q(s_{t+1}, a)$ assume controle ótimo futuro sem penalizar a rota pela exploração comportamental.
- **(b) Gulosa (-13) vs Avaliação $\epsilonhBcgulosa (-50 a -70):** Na rota gulosa determin ($\epsilon=0$), ele nunca cai. Mas na política $\epsilonhBcgulosa, como ele caminha na beirada, qualquer sorteio aleatório para o sul o derruba no penhasco (penalidade -100 frequente).

> 🔴 **O QUE PODE GERAR DÚVIDA / PEGADINHA:**
>
> Por que o Q-learning não tem 'medo' do penhasco como o SARSA? **Resposta:** Porque a polí alvo do Q-learning é $\max_a$ (gulosa pura). Ela não inclui ϵ. Ela assume que o age sempre tomará a decisão ótima a partir do próximo passo, ignorando desvios exploratórios.

Experimento: implemente `update_policy`, `get_target_value` e `update_Q` de `QLearningAgent` e o laço de `run_episode_and_train`, e execute a seção **Experiment** treinamento na configuração base, avaliação da política ϵ-gulosa e visualização.

Perguntas:

- **(a)** Compare a trajetória gulosa com o caminho mais curto (13 passos, rente ao penhasco). na forma como o Q-learning calcula o alvo TD $\bar q_t$, explica o caminho aprendido?
- **(b)** Compare a recompensa total da trajetória gulosa com a recompensa total média de uma ϵ-gulosa em relação aos valores de ação estimados, medida com `evaluate_po` Por que são diferentes?

Resposta

In [ ]:
```
df_base_eval = evaluate_policy(agent)
states, actions, rewards = simulate_trajectory(agent, ENV_ID)
table_base = pd.DataFrame([{
    "Configuração": "Baseline (α=0.1, γ=0.9, ε=0.1, N=1000)",
    "Chegada ao alvo (ε-gulosa)": df_base_eval["chegada ao alvo"],
    "Recompensa média (ε-gulosa)": df_base_eval["recompensa total média"]
    "Duração média (ε-gulosa)": df_base_eval["duração média"],
    "Passos (gulosa)": len(actions),
    "Recompensa (gulosa)": sum(rewards),
}])
display(table_base)
```

	Configuração	Chegada ao alvo (ε-gulosa)	Recompensa média (ε-gulosa)	Duração média (ε-gulosa)	Passos (gulosa)	Recompe (gulo
0	Baseline (α=0.1, γ=0.9, ε=0.1, N=1000)	1.0	-52.45	20.77	13	-1

- **(a)**

A trajetória gulosa obtida pelo agente coincide exatamente com o caminho mais curto possível r ambiente (13 passos e retorno total de -13): o agente sobe um passo, percorre 11 passos para a na linha 2 (colado à borda do penhasco) e desce no alvo. A explicação está na natureza off-poli learning e na forma como o alvo TD se comporta

O operador $\max_a$ assume que, a partir do próximo estado S o agente sempre executará a ação ignorando completamente o fato de que a política real que gera as ações no ambiente é epsilon (isto é, suscetível a desvios exploratórios). Como o cálculo do alvo desconsidera o risco de trope

abismo em passos futuros, o Q-learning estima diretamente os valores ótimos, convergindo para geometricamente mais curta.

- **(b)**

A recompensa total da trajetória gulosa é de -13, enquanto a recompensa média obtida com a p de epsilon-gulosa medida por evaluate_policy é muito mais baixa (na faixa de -50 a -70).

1. Trajetória gulosa (epsilon = 0): O agente age de forma puramente determinística, escolhend sempre a melhor ação. Como não há exploração, ele percorre a borda do penhasco com pr cirúrgica e nunca cai, acumulando apenas a penalidade de -1 por passo ao longo dos 13 pa
2. Política ε-gulosa (evaluate_policy): Em cada passo existe uma probabilidade epsilon = 0.1 d agente tomar uma ação aleatória. Como o caminho ótimo passa colado ao penhasco, basta único sorteio na direção "baixo" para o agente despencar, sofrer a severa penalidade de -1(obrigado a reiniciar a trajetória desde a origem. O acúmulo de quedas ocasionais derruba drasticamente a recompensa média.

1. Efeito da taxa de aprendizado (α)

> 🟢 **O QUE DETALHAR NA TAREFA 1 (alpha):**
>
> - **$ lpha=0.01$:** Muito lento para propagar em 1000 episódios.
> - **$ lpha=0.1$:** Taxa ideal que atinge 100% de sucesso e encontra a rota de 13 passos.
> - **$ lpha=0.5$ e .0$:** Aprendem muito rápido, mas sofrem de forte oscilação nas curvas de

> 🔴 **O QUE PODE GERAR DÚVIDA / PEGADINHA:**
>
> Por que com $ lpha=1.0$ a trajetória gulosa final ainda consegue chegar ao alvo em 13 pass
> **Resposta:** Porque o ambiente é determinístico nas transições físicas. Quando o agente finalr experimenta a melhor ação na beirada sem cair, com $ lpha=1.0$ ele grava esse valor instantaneamente.

Experimento: treine o Q-learning para $\alpha \in \{0.01,\ 0.1,\ 0.5,\ 1.0\}$, com um ambiente e agente novos para cada valor, fixando:

- $\gamma = 0.9$;
- $\epsilon = 0.1$;
- `n_episodes = 1000`;
- os demais parâmetros do baseline.

Crie os parâmetros de cada execução a partir do baseline com `_replace`:

```
params_exp = params._replace(learning_rate=0.5)
```

Perguntas:

- **(a)** Como α afetou o aprendizado?

Resposta

In [ ]:

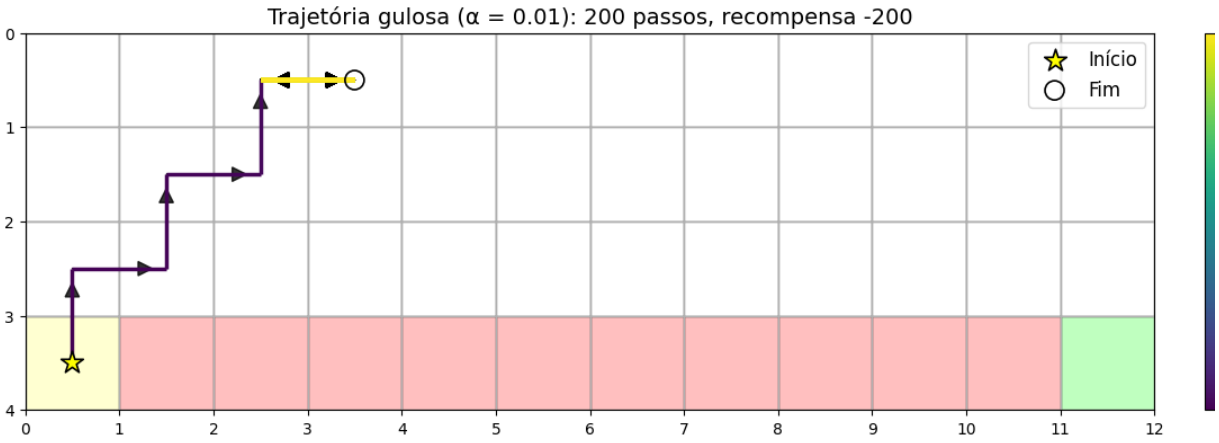
```
alphas = [0.01, 0.1, 0.5, 1.0]
histories_alpha = {}
results_alpha = []

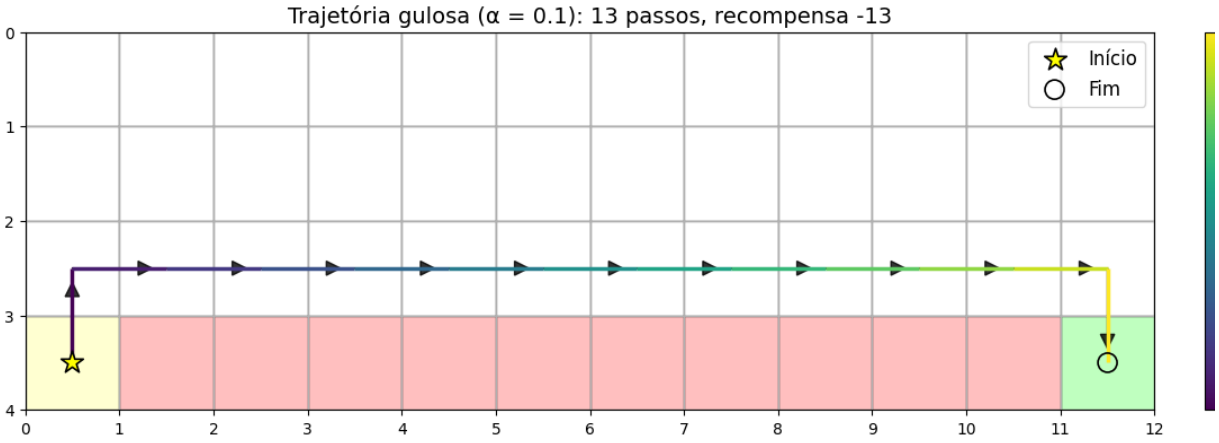
for alpha in alphas:
    label = f"α = {alpha}"
    params_exp = params._replace(learning_rate=alpha)
    agent_exp = QLearningAgent(params_exp)
    env_exp = make_env()
    history_exp = train_agent(agent_exp, env_exp, params_exp.n_episodes)
    histories_alpha[label] = history_exp

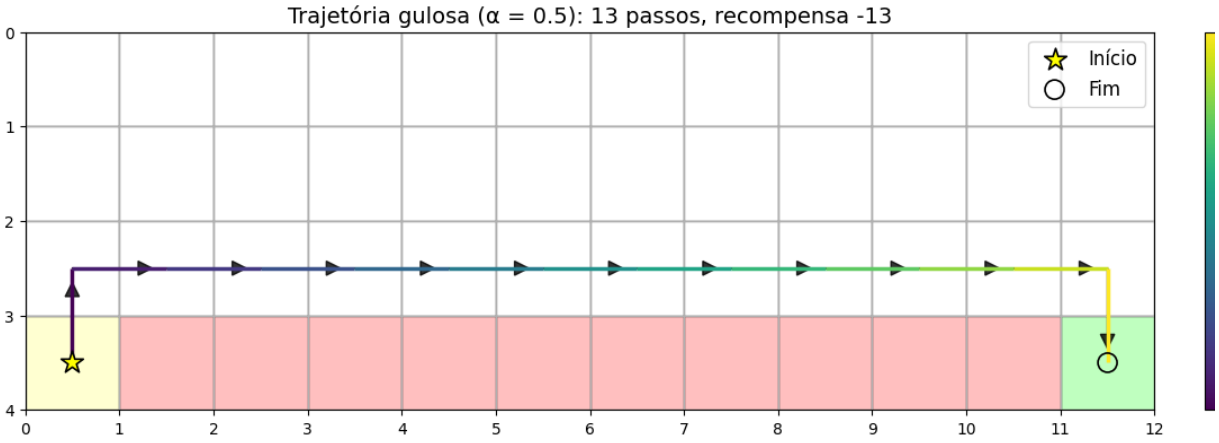
    eval_series = evaluate_policy(agent_exp)
    states, actions, rewards = simulate_trajectory(agent_exp, ENV_ID)
    plot_trajectory(ENV_ID, states, title=f"Trajetória gulosa ({label}):

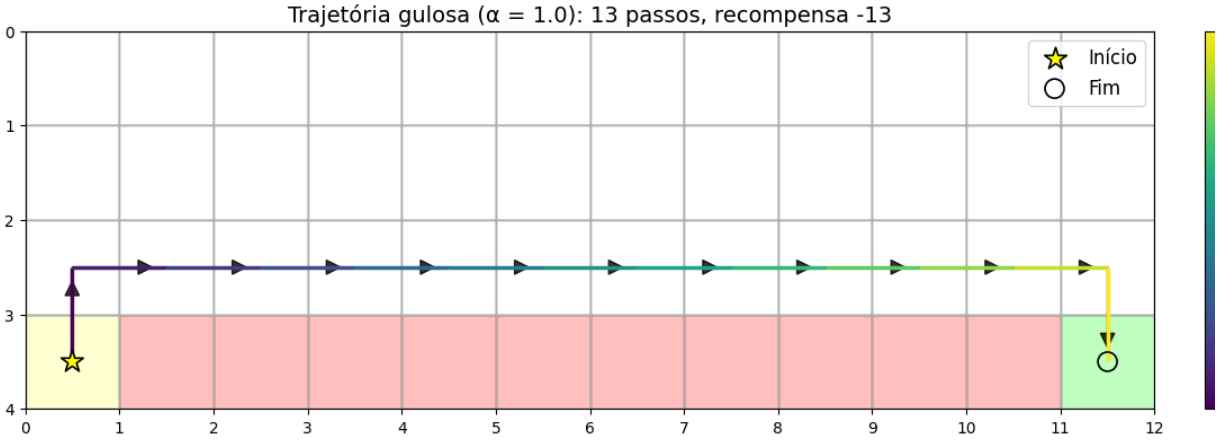
    results_alpha.append({
        "alpha": alpha,
        "Chegada ao alvo (ε-gulosa)": eval_series["chegada ao alvo"],
        "Recompensa média (ε-gulosa)": eval_series["recompensa total médi
        "Duração média (ε-gulosa)": eval_series["duração média"],
        "Passos (gulosa)": len(actions),
        "Recompensa (gulosa)": sum(rewards),
    })

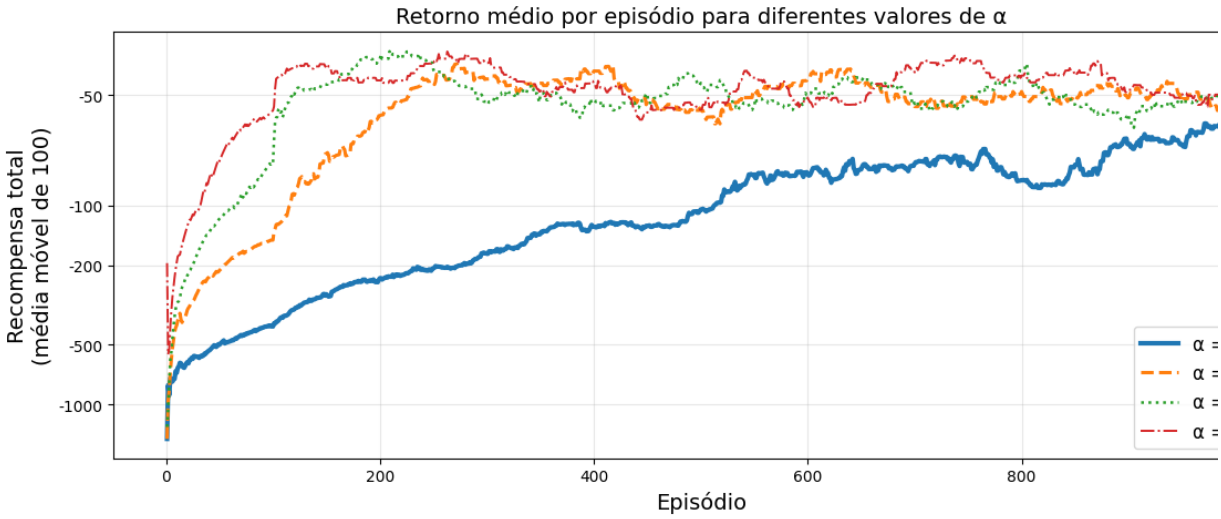
plot_comparison(histories_alpha, metric="returns", title="Retorno médio p
plot_comparison(histories_alpha, metric="lengths", title="Duração média p
df_alpha = pd.DataFrame(results_alpha)
display(df_alpha)
```

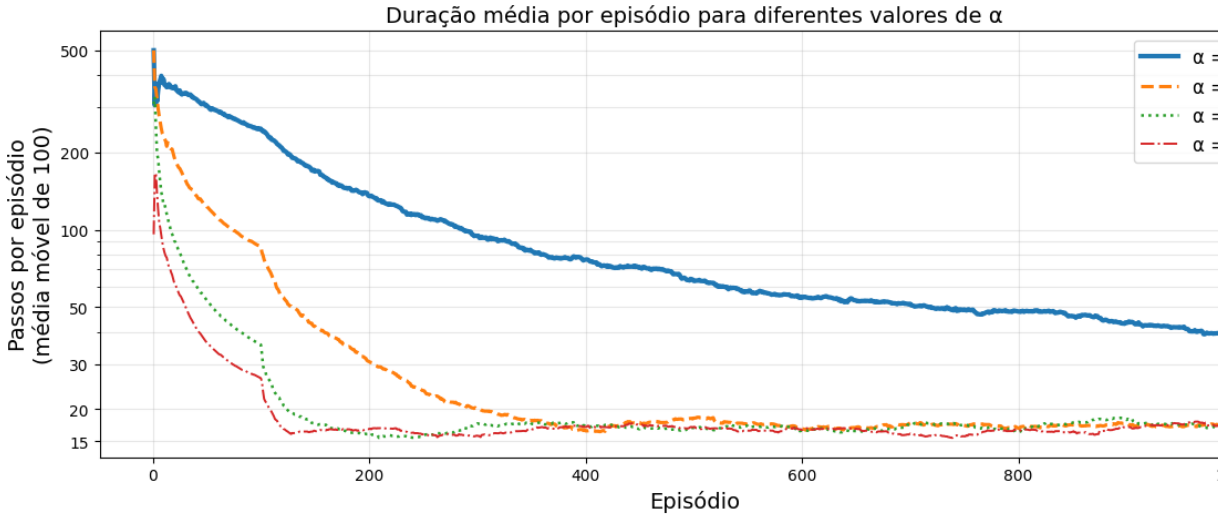

Trajetória gulosa (α = 0.01): 200 passos, recompensa -200
Início
Fim


Trajetória gulosa (α = 0.1): 13 passos, recompensa -13
Início
Fim


Trajetória gulosa (α = 0.5): 13 passos, recompensa -13
Início
Fim


Trajetória gulosa (α = 1.0): 13 passos, recompensa -13
Início
Fim





	alpha	Chegada ao alvo (ε-gulosa)	Recompensa média (ε-gulosa)	Duração média (ε-gulosa)	Passos (gulosa)	Recompe (gulo
0	0.01	0.07	-485.81	474.92	200	-20
1	0.10	1.00	-52.45	20.77	13	-1
2	0.50	1.00	-50.34	16.68	13	-1
3	1.00	1.00	-43.16	16.43	13	-1

- **(a)**

A taxa α dita o peso que o agente dá para a nova evidência:

• α = 0.01 estando muito baixo: O aprendizado é excessivamente lento. A cada passo, a estimat quase nada. Em 1000 episódios, o agente não tem tempo suficiente para propagar os valores d até a origem, resultando em subconvergência.

• α = 0.1 estnado em um nivel medio baseline ideal: Se torna equilíbra do perfeito entre velocida estabilidade. O agente aprende com suavidade, filtra ruídos e converge com segurança para a p ótima de 13 passos.

• α = 0.5 e α = 1.0 muito altos: O aprendizado inicial parece rápido, mas torna-se extremamente e ruidoso. Em α = 1.0, o agente substitui bruscamente o valor antigo pelo novo alvo a cada pass

o ambiente possui exploração estocástica, as curvas de recompensa sofrem oscilações violenta estabilizam

2. Efeito do fator de desconto (γ)

> 🟢 **O QUE DETALHAR NA TAREFA 2 (gamma):**
>
> - **$\gamma=0$: Miopia Absoluta:** O agente só se importa com {t+1}$. Como todo passo nenhuma ação se destaca e ele fica em loop até 500 passos (truncamento).
> - **$\gamma=0.5$:** O desconto é forte demais para 13 passos futuros (bash.5^{13} pprox 0.0001$). A recompensa de chegar ao alvo fica praticamente invisível no início.
> - **$\gamma=0.9$ e bash.99$:** Visão de longo prazo perfeita, descobrindo o caminho de 13 passos.

> 🔴 **O QUE PODE GERAR DÚVIDA / PEGADINHA:**
>
> Qual a diferença de $\gamma=0.5$ no Q-learning vs SARSA? **Resposta:** Em ambos, $ \gamma=0.5$ falha pela miopia, mas no Q-learning o truncamento acontece porque a retropropagação do valor do alvo atenua exponencialmente pela distância de 13 passos.

Experimento: treine o Q-learning para $\gamma \in \{0,\ 0.5,\ 0.9,\ 0.99\}$, com um ambiente e agente novos para cada valor, fixando:

- $\alpha = 0.1$;
- $\epsilon = 0.1$;
- `n_episodes = 1000`;
- os demais parâmetros do baseline.

Perguntas:

- **(a)** Como γ afetou a trajetória gulosa?

Resposta

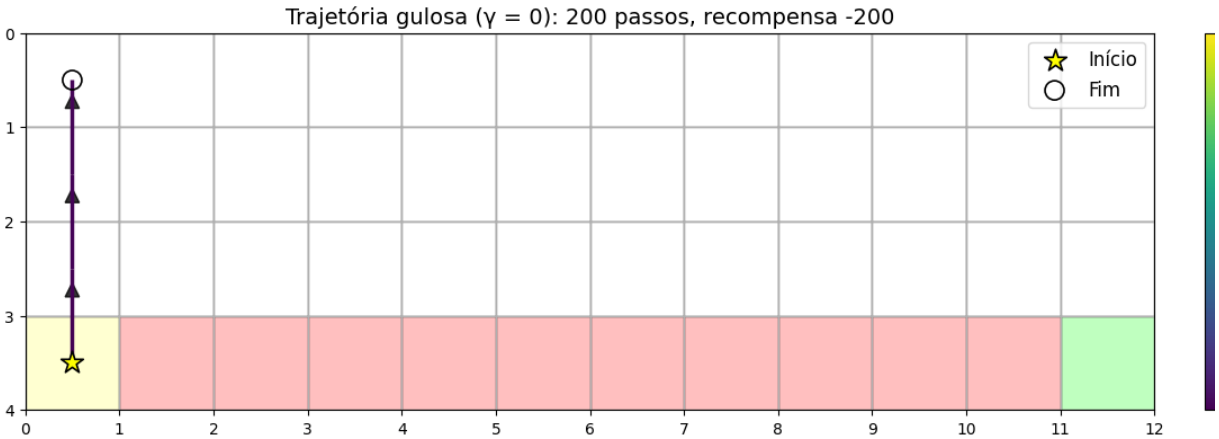
In [ ]:
```
gammas = [0, 0.5, 0.9, 0.99]
histories_gamma = {}
results_gamma = []

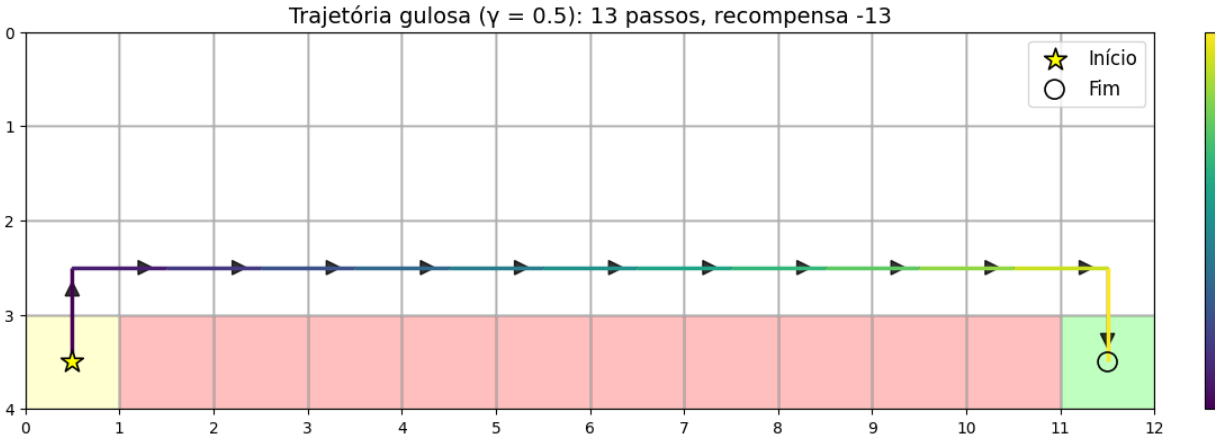
for gamma in gammas:
    label = f"γ = {gamma}"
    params_exp = params._replace(discount_factor=gamma)
    agent_exp = QLearningAgent(params_exp)
    env_exp = make_env()
    history_exp = train_agent(agent_exp, env_exp, params_exp.n_episodes)
    histories_gamma[label] = history_exp

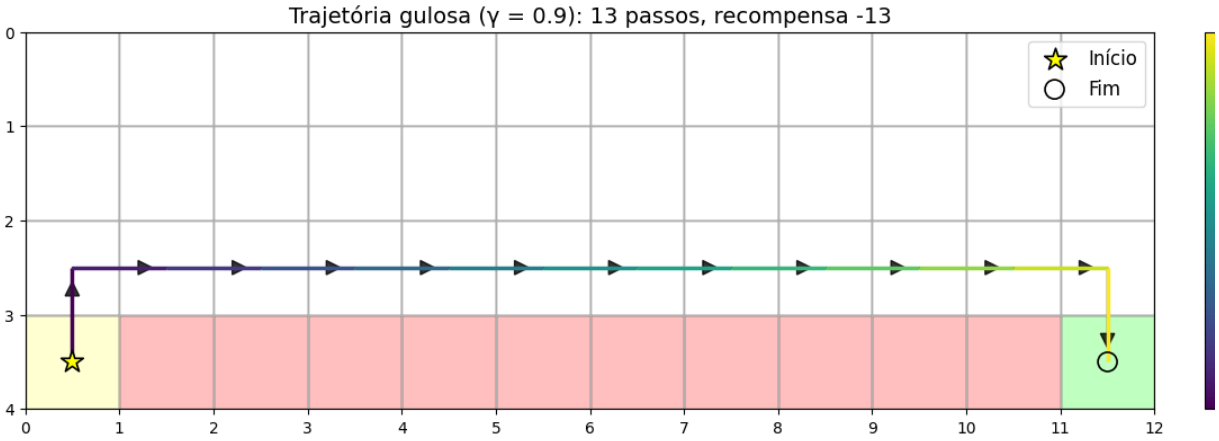
    eval_series = evaluate_policy(agent_exp)
    states, actions, rewards = simulate_trajectory(agent_exp, ENV_ID)
    plot_trajectory(ENV_ID, states, title=f"Trajetória gulosa ({label}):
```

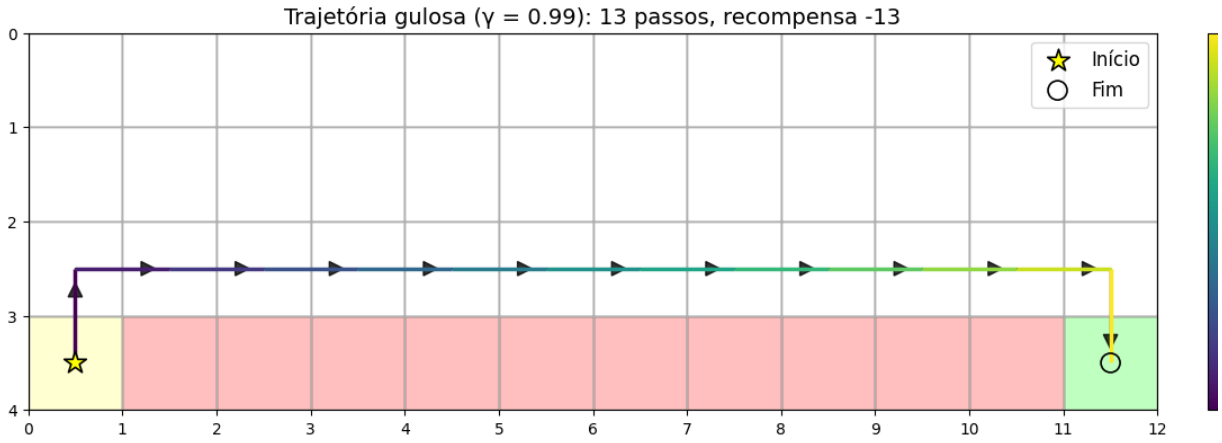
```
    results_gamma.append({
        "gamma": gamma,
        "Chegada ao alvo (ε-gulosa)": eval_series["chegada ao alvo"],
        "Recompensa média (ε-gulosa)": eval_series["recompensa total médi
        "Duração média (ε-gulosa)": eval_series["duração média"],
        "Passos (gulosa)": len(actions),
        "Recompensa (gulosa)": sum(rewards),
    })

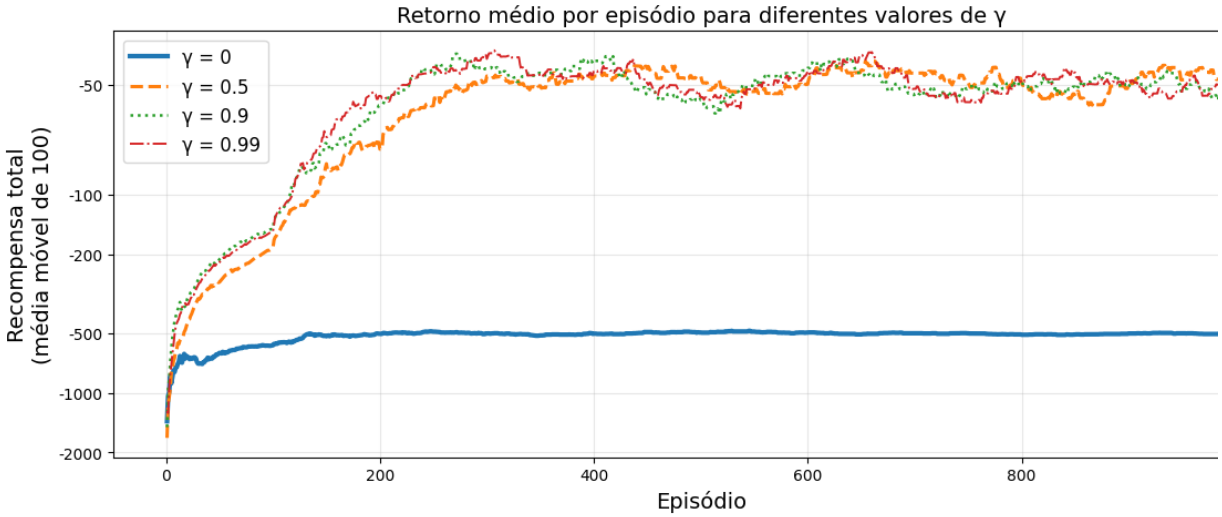
plot_comparison(histories_gamma, metric="returns", title="Retorno médio p
plot_comparison(histories_gamma, metric="lengths", title="Duração média p
df_gamma = pd.DataFrame(results_gamma)
display(df_gamma)
```













	gamma	Chegada ao alvo (ε-gulosa)	Recompensa média (ε-gulosa)	Duração média (ε-gulosa)	Passos (gulosa)	Recompe (gulo
0	0.00	0.01	-502.98	499.02	200	-20
1	0.50	1.00	-47.37	16.68	13	-1
2	0.90	1.00	-52.45	20.77	13	-1
3	0.99	1.00	-55.25	20.60	13	-1

- **(a)**

O desconto γ define o quanto o agente valoriza o futuro em relação ao presente sendo assim;

• Para γ = 0 o agente tem miopia absoluta: Só se importa com a recompensa imediata r. Como t passo no chão custa -1, qualquer direção parece igualmente ruim para ele. O agente é incapaz enxergar que chegar ao objetivo no futuro trará recompensas melhores e simplesmente vagueia rumo até o episódio ser truncado (não atinge o alvo).

• γ ≥ 0.5 (0.5, 0.9, 0.99): O agente consegue enxergar a meta e retropropagar os valores de açã todos esses casos, o agente consegue aprender a trajetória ótima de 13 passos. Quanto mais p de 1.0 (γ = 0.9 e 0.99), mais nítido e suave é o gradiente de valores de estado ao longo de toda

3. Efeito do parâmetro ϵ

> 🟢 **O QUE DETALHAR NA TAREFA 3 (epsilon):**
>
> - **Trajetória Gulosa invariante:** Para todos os valores de ϵ, a rota gulosa é a ME 13 passos ótimos na beirada (-13). Isso comprova que o Q-learning aprende *$ desacopl política comportamental!
> - **Recompensa com evaluate_policy:** Despenca violentamente com $\epsilon=0.3$, pois exploração significa quedas contínuas no penhasco.

> 🔴 **O QUE PODE GERAR DÚVIDA / PEGADINHA:**
>
> Por que no SARSA o ϵ altera a trajetória gulosa, mas no Q-learning NÃO altera? **Re crucial:** Porque o SARSA é *on-policy* (seu alvo muda com ϵ). O Q-learning é *off-pol* alvo é sempre $\max_a$, independente de ϵ).

Experimento: treine o Q-learning para $\epsilon \in \{10^{-3},\ 10^{-2},\ 10^{-1},\ 0.3\}$, com um ambiente e um agente novos para cada valor, fixando:

- $\alpha = 0.1$;
- $\gamma = 0.9$;
- `n_episodes = 1000`;
- os demais parâmetros do baseline.

Perguntas:

- **(a)** Como ϵ afetou a trajetória gulosa e a recompensa total média da política $\eps gulosa, medida com `evaluate_policy`? Explique por quê.

Resposta

In [ ]:
```
epsilons = [1e-3, 1e-2, 1e-1, 0.3]
histories_eps = {}
results_eps = []

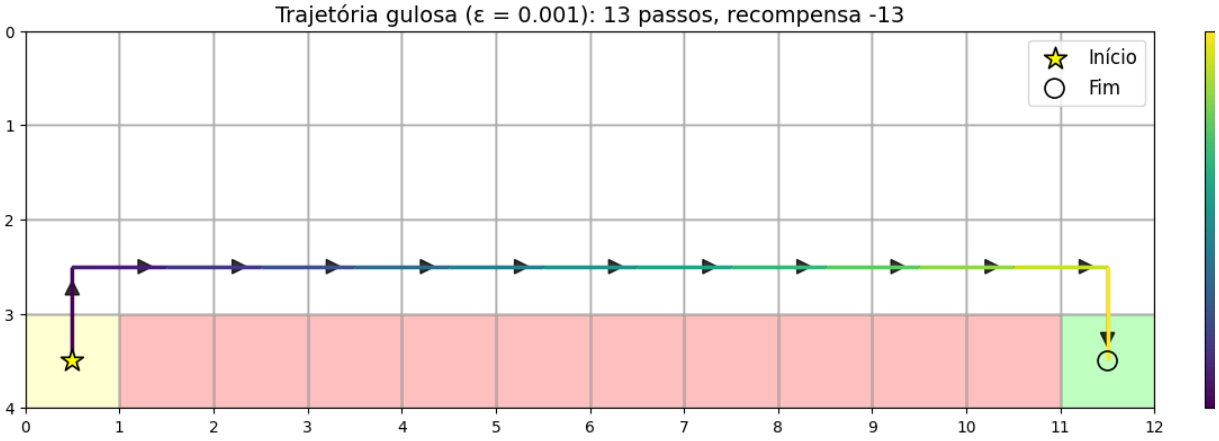
for eps in epsilons:
    label = f"ε = {eps}"
    params_exp = params._replace(epsilon=eps)
    agent_exp = QLearningAgent(params_exp)
```

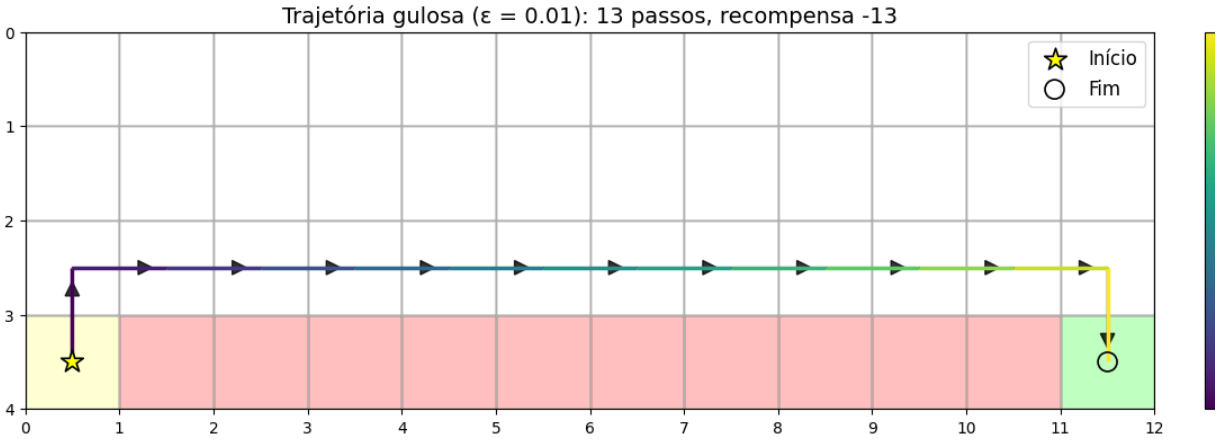
```
    env_exp = make_env()
    history_exp = train_agent(agent_exp, env_exp, params_exp.n_episodes)
    histories_eps[label] = history_exp

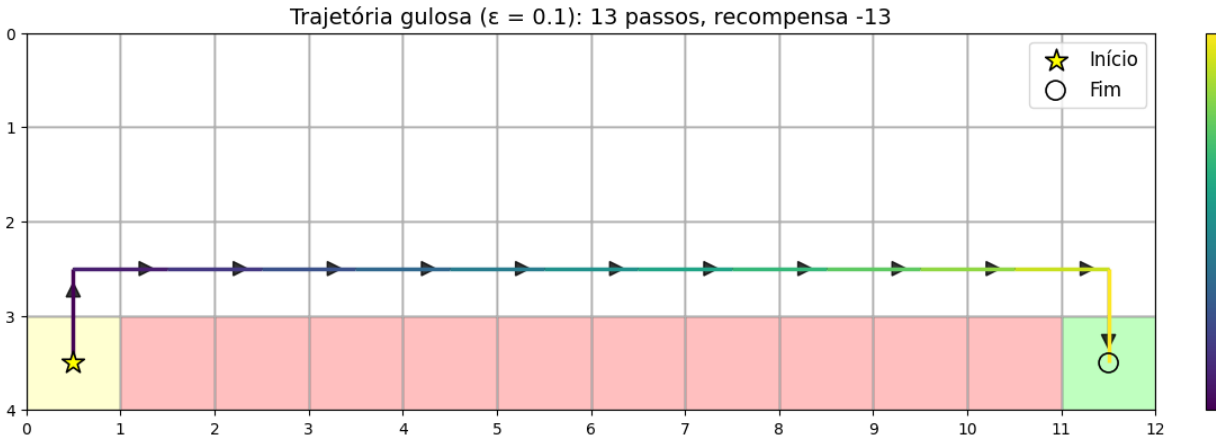
    eval_series = evaluate_policy(agent_exp)
    states, actions, rewards = simulate_trajectory(agent_exp, ENV_ID)
    plot_trajectory(ENV_ID, states, title=f"Trajetória gulosa ({label}):

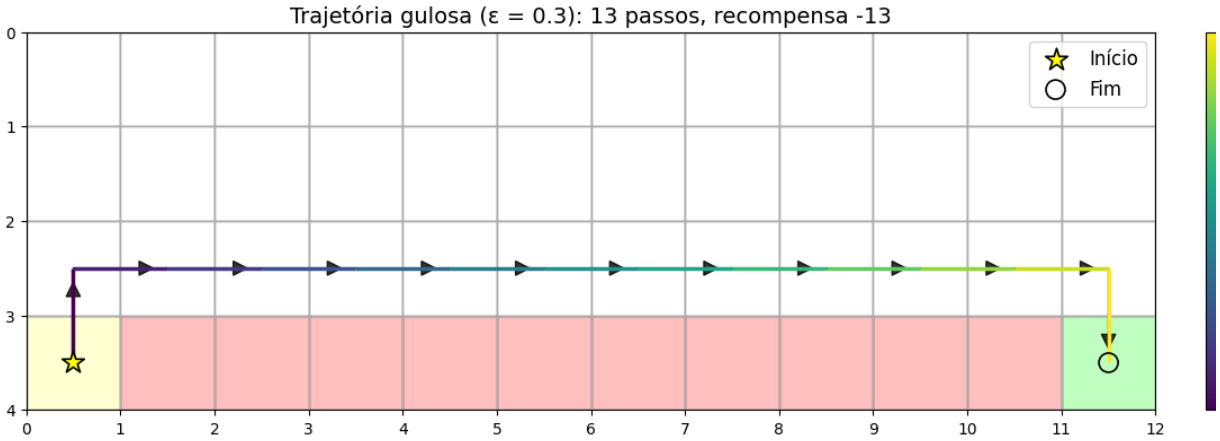
    results_eps.append({
        "epsilon": eps,
        "Chegada ao alvo (ε-gulosa)": eval_series["chegada ao alvo"],
        "Recompensa média (ε-gulosa)": eval_series["recompensa total médi
        "Duração média (ε-gulosa)": eval_series["duração média"],
        "Passos (gulosa)": len(actions),
        "Recompensa (gulosa)": sum(rewards),
    })

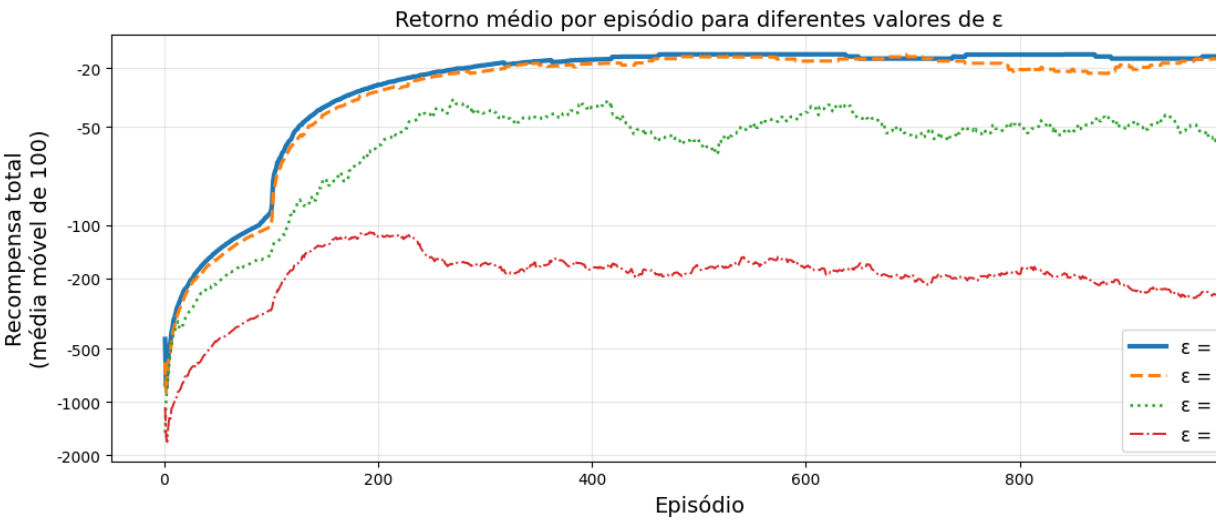
plot_comparison(histories_eps, metric="returns", title="Retorno médio por
plot_comparison(histories_eps, metric="lengths", title="Duração média por
df_eps = pd.DataFrame(results_eps)
display(df_eps)
```

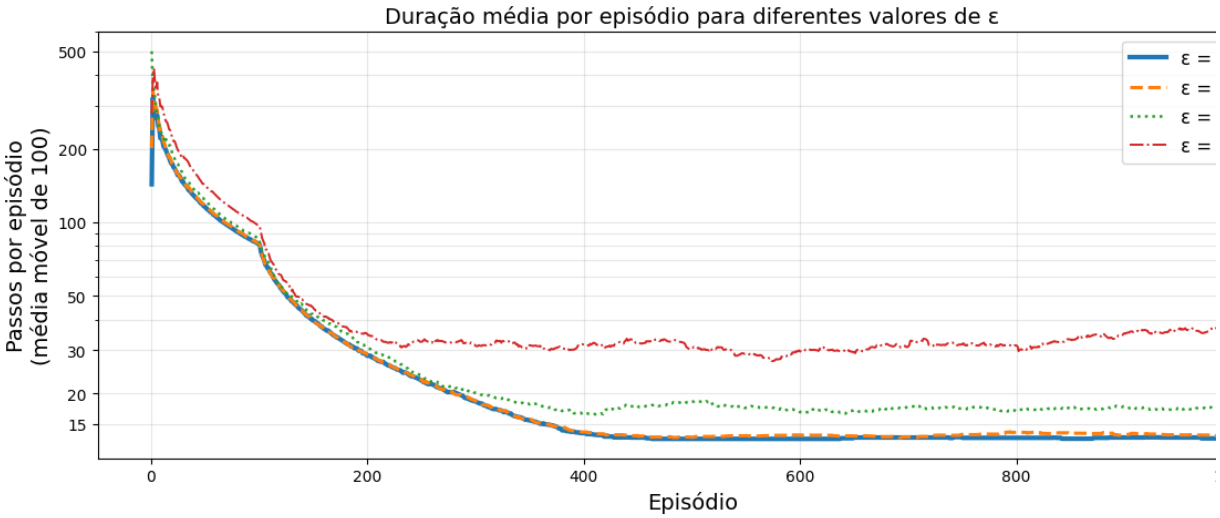





Trajetória gulosa (ε = 0.1): 13 passos, recompensa -13
Início
Fim


Trajetória gulosa (ε = 0.3): 13 passos, recompensa -13
Início
Fim


Retorno médio por episódio para diferentes valores de ε
Recompensa total
(média móvel de 100)
Episódio
ε =
ε =
ε =
ε =


Duração média por episódio para diferentes valores de ε
Passos por episódio
(média móvel de 100)
Episódio
ε =
ε =
ε =
ε =

	epsilon	Chegada ao alvo (ε-gulosa)	Recompensa média (ε-gulosa)	Duração média (ε-gulosa)	Passos (gulosa)	Recompe (gulo
0	0.001	1.00	-13.00	13.00	13	-1
1	0.010	0.99	-22.21	18.25	13	-1
2	0.100	1.00	-52.45	20.77	13	-1
3	0.300	1.00	-171.31	31.72	13	-1

- **(a)**

O parâmetro epsilon afeta a taxa de exploração:

- **Trajetória gulosa:** Para todos os valores de epsilon testados, a trajetória gulosa é a mesma caminho ótimo de 13 passos (recompensa hBc13). Isso ocorre porque o Q-learning é off-po aprende a função de valor ótima, desacoplada da exploração.
- **Recompensa média com :** À medida que epsilon aumenta, a recompensa média despenc drasticamente tornando-se muito mais negativa. Como o caminho ótimo passa colado ao pe quanto maior epsilon, maior a probabilidade de o agente sortear uma ação para baixo durar avaliação e cair no abismo, gerando quedas frequentes e penalidades pesadas de hBc100.

4. Efeito do número de episódios (N)

🟢 **O QUE DETALHAR NA TAREFA 4 (N):**

- **=1$ e =10$:** Falham completamente (0% de sucesso). O agente ainda não visitou o alvo suficientes para propagar os valores até o estado inicial.
- **=100$:** Começa a aprender, mas ainda de forma subótima.
- **=1000$:** Consolida o caminho ótimo estrito de 13 passos.

🔴 **O QUE PODE GERAR DÚVIDA / PEGADINHA:**

Por que o agente precisa de 1000 episódios se a grade só tem 48 estados? **Resposta:** Porqu exploração é cega e aleatória no início. A maioria dos primeiros episódios termina em quedas imediatas no penhasco ou esgotando o limite de 500 passos sem atingir o alvo.

Experimento: treine o Q-learning para `n_episodes` $\in \{1,\ 10,\ 100,\ 1000\}$, com um amb
um agente novos para cada valor, fixando:

- $\alpha = 0.1$;
- $\gamma = 0.9$;
- $\epsilon = 0.1$;
- os demais parâmetros do baseline.

Perguntas:

- **(a)** Para quais valores de N a trajetória gulosa chega ao alvo? Explique esse comportame

Resposta

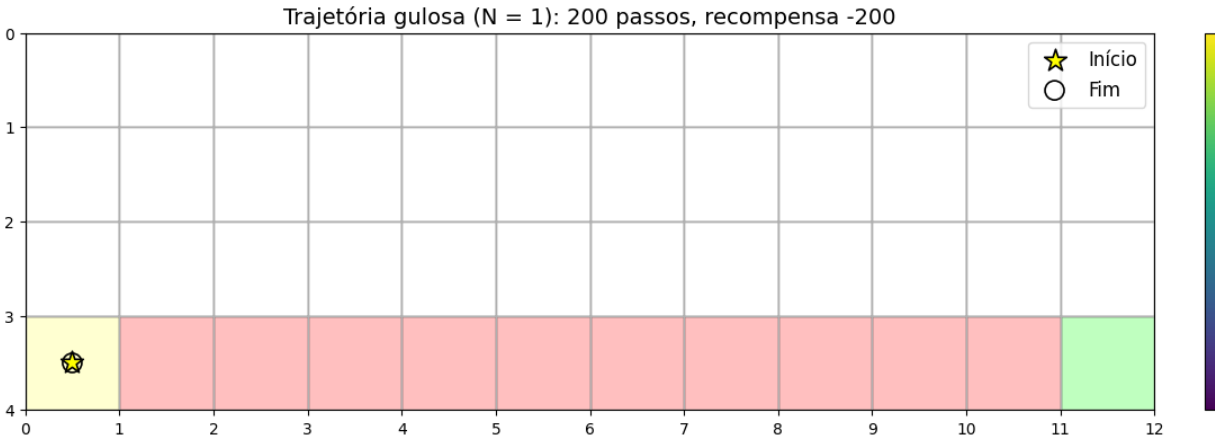
In [ ]:
```
n_episodes_list = [1, 10, 100, 1000]
histories_n = {}
results_n = []

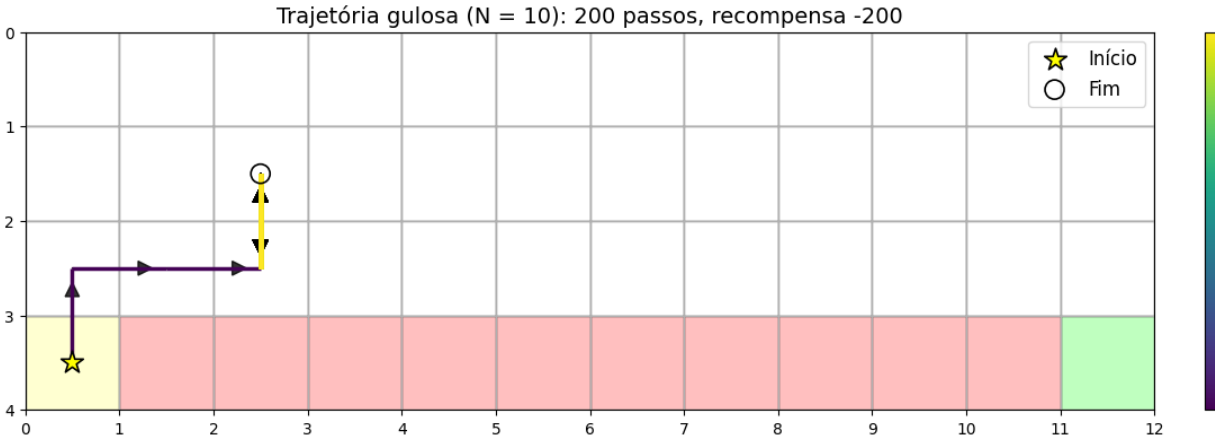
for n_ep in n_episodes_list:
    label = f"N = {n_ep}"
    params_exp = params._replace(n_episodes=n_ep)
    agent_exp = QLearningAgent(params_exp)
    env_exp = make_env()
    history_exp = train_agent(agent_exp, env_exp, params_exp.n_episodes)
    histories_n[label] = history_exp

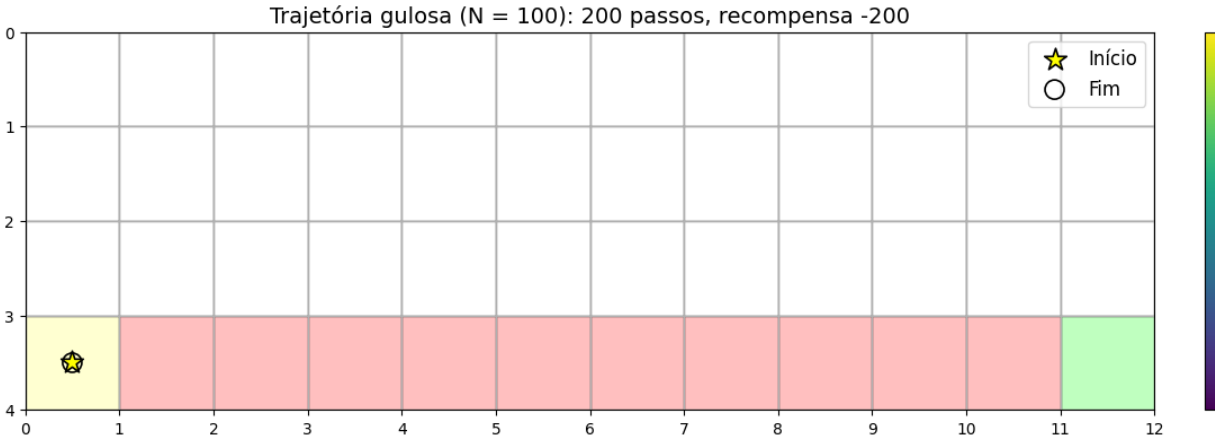
    eval_series = evaluate_policy(agent_exp)
    states, actions, rewards = simulate_trajectory(agent_exp, ENV_ID)
    plot_trajectory(ENV_ID, states, title=f"Trajetória gulosa ({label}):

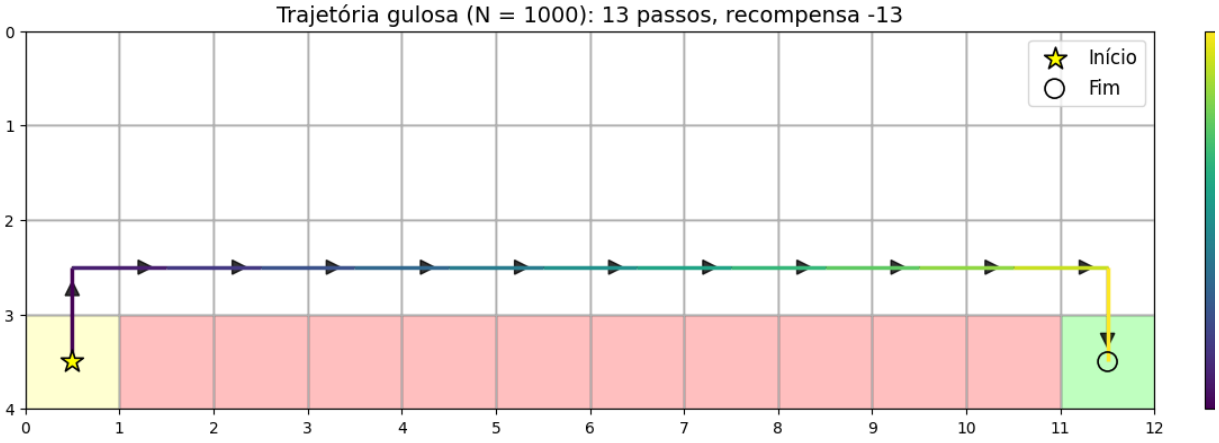
    results_n.append({
        "N (episódios)": n_ep,
        "Chegada ao alvo (ε-gulosa)": eval_series["chegada ao alvo"],
        "Recompensa média (ε-gulosa)": eval_series["recompensa total médi
        "Duração média (ε-gulosa)": eval_series["duração média"],
        "Passos (gulosa)": len(actions),
        "Recompensa (gulosa)": sum(rewards),
    })

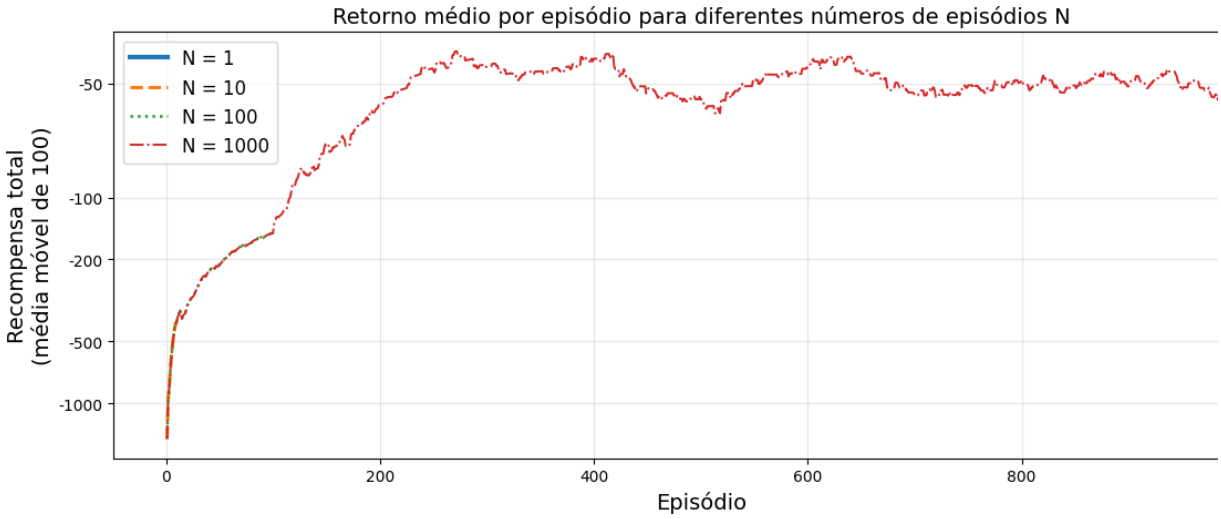
plot_comparison(histories_n, metric="returns", title="Retorno médio por e
plot_comparison(histories_n, metric="lengths", title="Duração média por e
df_n = pd.DataFrame(results_n)
display(df_n)
```


Trajetória gulosa (N = 1): 200 passos, recompensa -200
Início
Fim


Trajetória gulosa (N = 10): 200 passos, recompensa -200
Início
Fim


Trajetória gulosa (N = 100): 200 passos, recompensa -200
Início
Fim


Trajetória gulosa (N = 1000): 13 passos, recompensa -13
Início
Fim





	N (episódios)	Chegada ao alvo (ε-gulosa)	Recompensa média (ε-gulosa)	Duração média (ε-gulosa)	Passos (gulosa)	Recompe (gulo
0	1	0.00	-1695.92	500.00	200	-2(
1	10	0.00	-578.21	500.00	200	-2(
2	100	0.24	-892.96	451.42	200	-2(
3	1000	1.00	-52.45	20.77	13	-

- **(a)**

A trajetória gulosa chega ao alvo apenas para = 100 e = 1000 atingindo o caminho ótimo de 13 em = 1000.

- **Explicação:** Para = 1 e = 10, o agente visitou pouquíssimos pares estado-ação e ainda nã alcançou o alvo ou o alcançou pouquíssimas vezes, de modo que o erro TD não teve episó suficientes para retropropagar os valores de recompensa até o estado inicial. Sem valores informativos na tabela, a política gulosa permanece presa em ações padrão e o episódio te truncamento.

5. Comparação com o Sarsa

> 🟢 **O QUE DETALHAR NA TAREFA 5 (O GRANDE DUELO):**
>
> - **Com $\epsilon=0.1$ em Treino/Online:** O **SARSA é muito superior** (retorno médio ent
> e -30 vs -50 a -70 do Q-learning). O SARSA aprendeu a contornar por cima, sofrendo pou
> quedas ao explorar.
> - **Com $\epsilon=0.1$ na Trajetória Gulosa:** O **Q-learning é superior** (13 passos, retorn
> vs 15 passos, retorno -15 do SARSA).
> - **Quando $\epsilon o 0$:** Conforme ϵ diminui, o risco de quedas aleatórias desa
> O SARSA perde o medo do penhasco e sua trajetória gulosa converge exatamente para
> passos do Q-learning!

> 🔴 **O QUE PODE GERAR DÚVIDA / PEGADINHA DA BANCA:**
>
> *"Então o Q-learning é um algoritmo ruim porque tem curvas de recompensa piores durante o*
> **Resposta perfeita:** Não! O Q-learning cumpriu com perfeição seu objetivo matemático: conv
> para a política ótima estrita *$. Sua curva de treino é pior apenas porque ele foi forçado a usa
> política comportamental com exploração em um ambiente com penalidades catastróficas na b
> No deploy final (modo guloso determinístico), ele é o campeão.

Experimento: compare o Q-learning com o Sarsa do Laboratório 7 para os quatro valores de $\
do item 3:

- copie para a célula de resposta a classe `SarsaAgent` e a função `run_episode_and_t`
do seu notebook do Laboratório 7, renomeando a função para `run_episode_sarsa`, po
nome `run_episode_and_train` já é o do laço do Q-learning;
- para cada valor de ϵ, treine o Sarsa com um ambiente e um agente novos, passar
laço dele a `train_agent` pelo argumento `run_episode`:

```
history_sarsa = train_agent(sarsa_agent, make_env(),
params_exp.n_episodes,
                            run_episode=run_episode_sarsa)
```

- compare com as execuções do Q-learning do item 3, que usam os mesmos hiperparâmetro
\alpha = 0.1$, $\gamma = 0.9$, `n_episodes = 1000` e os demais parâmetros do baseli

Perguntas:

- **(a)** Com $\epsilon = 0.1$, qual dos dois algoritmos tem a maior recompensa total por episó
curvas de treinamento e com `evaluate_policy`? E qual tem a trajetória gulosa de maio
recompensa total? O que, no alvo TD de cada algoritmo, explica esse resultado?
- **(b)** Como a comparação entre os dois algoritmos muda à medida que ϵ diminui? F

Resposta

In [ ]:
```
class SarsaAgent:
    def __init__(self, params: Params, seed: int = SEED):
        self.n_states = params.n_states
        self.n_actions = params.n_actions
```

```
        self.learning_rate = params.learning_rate
        self.discount_factor = params.discount_factor
        self.epsilon = params.epsilon
        self.rng = np.random.default_rng(seed)
        shape = (self.n_states, self.n_actions)
        self.Q = np.zeros(shape)
        self.Pi = np.full(shape, 1.0 / self.n_actions)
        self.visit_counts = np.zeros(shape)

    def get_greedy_action(self, obs: int) -> int:
        return int(np.argmax(self.Q[obs]))

    def update_policy(self, obs: int) -> None:
        greedy_action = self.get_greedy_action(obs)
        base = self.epsilon / self.n_actions
        calc = (1.0 - self.epsilon) + base
        self.Pi[obs] = base
        self.Pi[obs, greedy_action] = calc

    def sample_action(self, obs: int) -> Tuple[int, int]:
        action = int(self.rng.choice(self.n_actions, p=self.Pi[obs]))
        is_exploratory = int(action != self.get_greedy_action(obs))
        return action, is_exploratory

    def policy(self, obs: int, deterministic: bool = False) -> int:
        if deterministic:
            return self.get_greedy_action(obs)
        return self.sample_action(obs)[0]

    def get_target_value(self, next_obs: int, next_action: int, reward: f
        if terminated:
            return reward
        return reward + self.discount_factor * self.Q[next_obs, next_acti

    def update_Q(self, obs: int, action: int, next_obs: int, next_action:
        target_value = self.get_target_value(next_obs, next_action, rewar
        td_error = self.Q[obs, action] - target_value
        self.Q[obs, action] -= self.learning_rate * td_error
        return td_error

    def update(self, obs: int, action: int, next_obs: int, next_action: i
        self.update_Q(obs, action, next_obs, next_action, reward, termina
        self.update_policy(obs)
        self.visit_counts[obs, action] += 1


def run_episode_sarsa(agent: SarsaAgent, env: gym.Env, seed: int = SEED):
    obs, _info = env.reset(seed=seed)
    action, is_exploratory = agent.sample_action(obs)
    n_exploratory_actions = is_exploratory
    episode_over = False
    terminated = False
    info = {}

    while not episode_over:
        next_obs, reward, terminated, truncated, info = env.step(action)
        next_action, is_exploratory = agent.sample_action(next_obs)
        agent.update(obs, action, next_obs, next_action, reward, terminat
        n_exploratory_actions += is_exploratory
        obs, action = next_obs, next_action
```

```
        episode_over = terminated or truncated

    return info["episode"], n_exploratory_actions, terminated


epsilons = [1e-3, 1e-2, 1e-1, 0.3]
results_sarsa_vs_q = []

for eps in epsilons:
    params_exp = params._replace(epsilon=eps)

    sarsa_agent = SarsaAgent(params_exp)
    history_sarsa = train_agent(sarsa_agent, make_env(), params_exp.n_epi

    q_agent = QLearningAgent(params_exp)
    history_q = train_agent(q_agent, make_env(), params_exp.n_episodes)

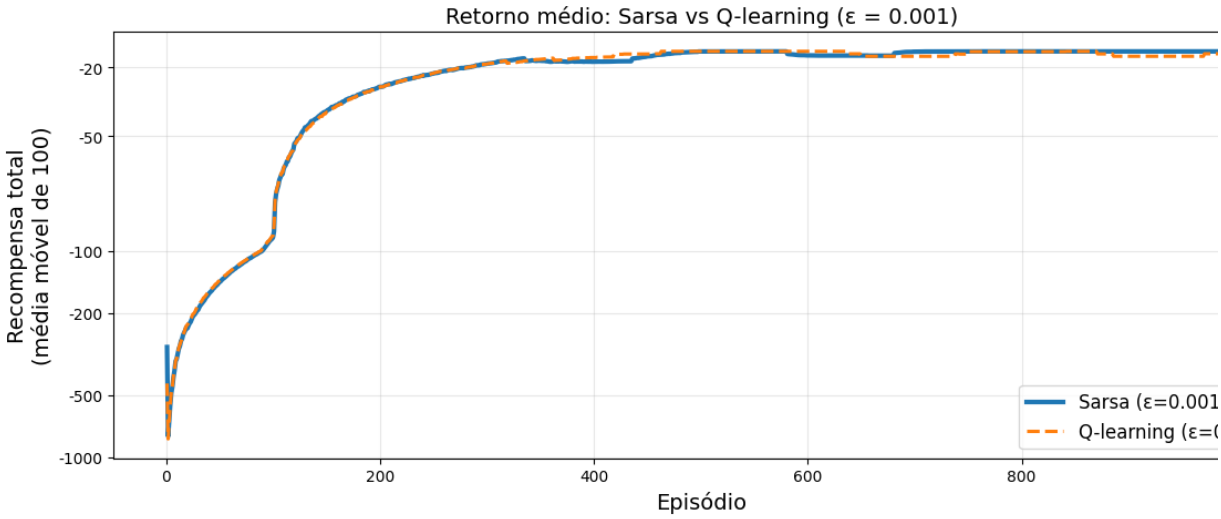
    comp_hist = {
        f"Sarsa (ε={eps})": history_sarsa,
        f"Q-learning (ε={eps})": history_q,
    }
    plot_comparison(comp_hist, metric="returns", title=f"Retorno médio: S
    plot_comparison(comp_hist, metric="lengths", title=f"Duração média: S

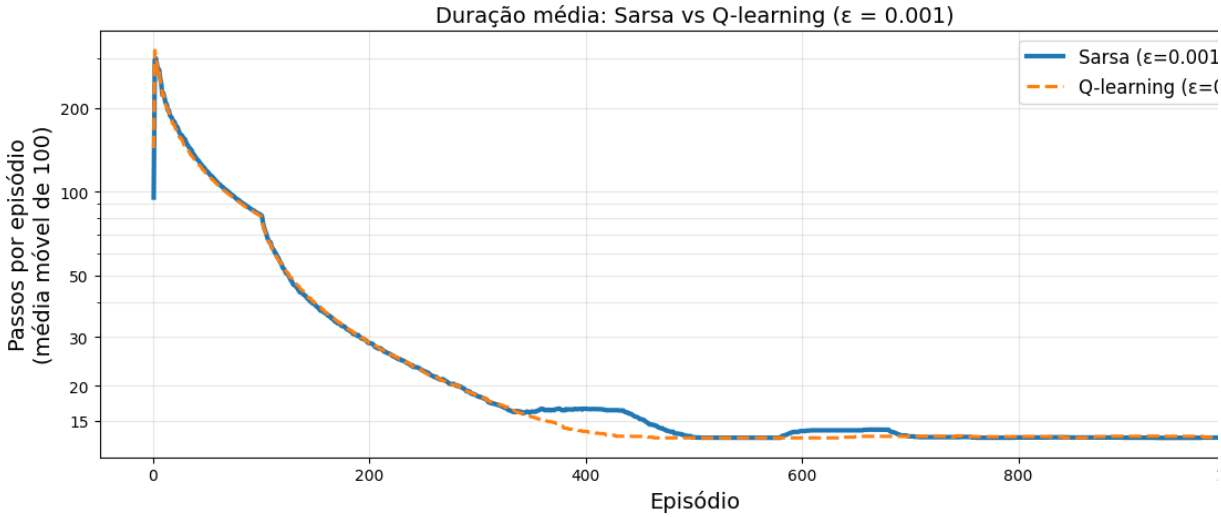
    st_s, act_s, rew_s = simulate_trajectory(sarsa_agent, ENV_ID)
    st_q, act_q, rew_q = simulate_trajectory(q_agent, ENV_ID)
    plot_trajectory(ENV_ID, st_s, title=f"Sarsa (ε={eps}): {len(act_s)} p
    plot_trajectory(ENV_ID, st_q, title=f"Q-learning (ε={eps}): {len(act_

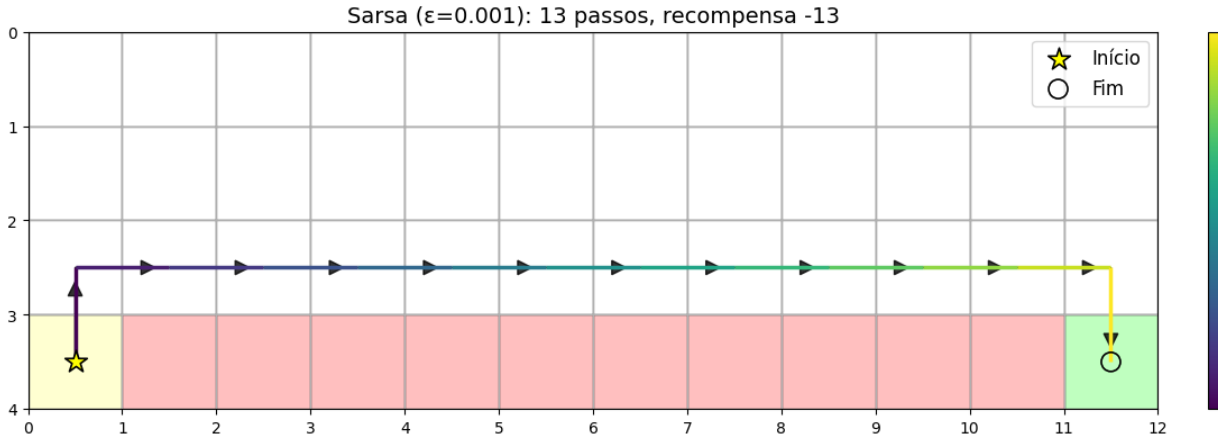
    eval_sarsa = evaluate_policy(sarsa_agent)
    eval_q = evaluate_policy(q_agent)

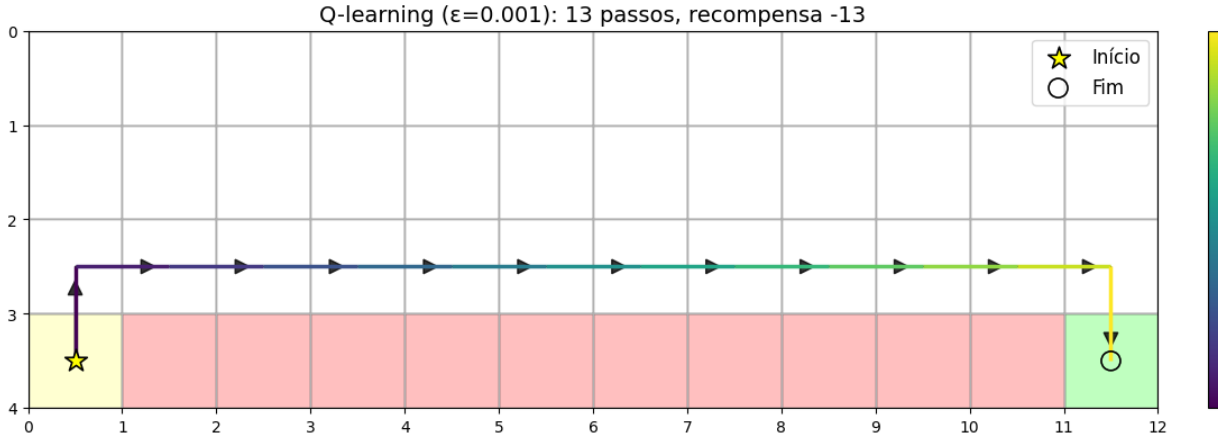
    results_sarsa_vs_q.append({
        "Algoritmo": "Sarsa",
        "epsilon": eps,
        "Chegada ao alvo (ε-gulosa)": eval_sarsa["chegada ao alvo"],
        "Recompensa média (ε-gulosa)": eval_sarsa["recompensa total média
        "Duração média (ε-gulosa)": eval_sarsa["duração média"],
        "Passos (gulosa)": len(act_s),
        "Recompensa (gulosa)": sum(rew_s),
    })
    results_sarsa_vs_q.append({
        "Algoritmo": "Q-learning",
        "epsilon": eps,
        "Chegada ao alvo (ε-gulosa)": eval_q["chegada ao alvo"],
        "Recompensa média (ε-gulosa)": eval_q["recompensa total média"],
        "Duração média (ε-gulosa)": eval_q["duração média"],
        "Passos (gulosa)": len(act_q),
        "Recompensa (gulosa)": sum(rew_q),
    })

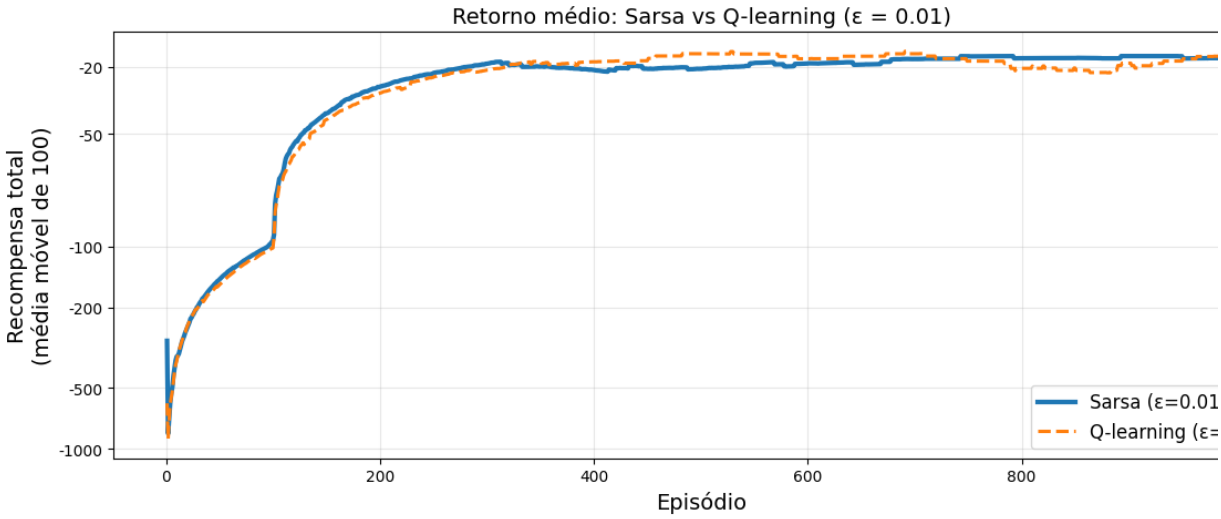
df_comp = pd.DataFrame(results_sarsa_vs_q)
display(df_comp)
```

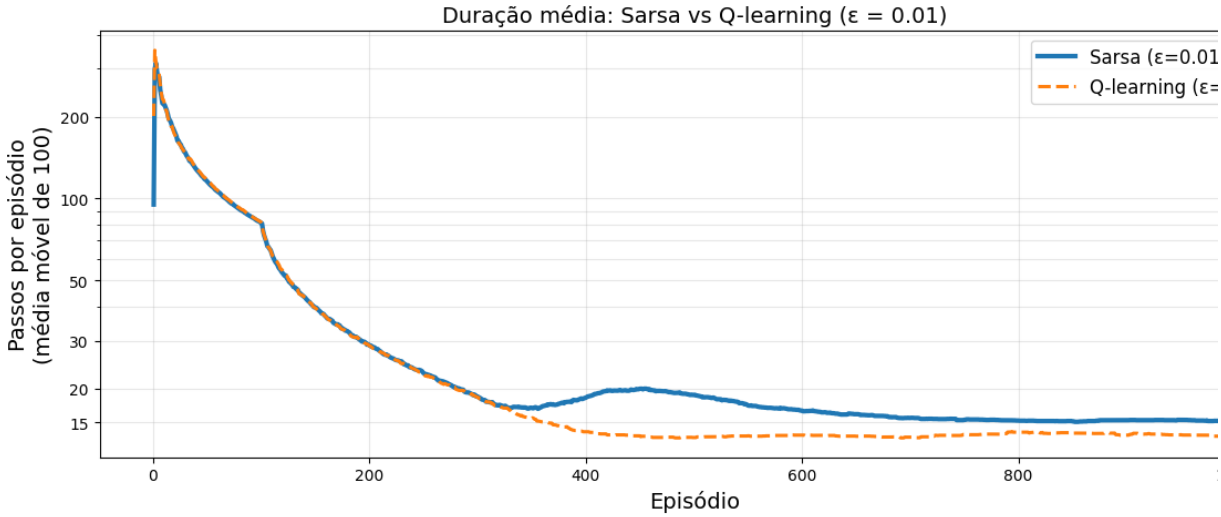

Retorno médio: Sarsa vs Q-learning (ε = 0.001)
Recompensa total
(média móvel de 100)
-20
-50
-100
-200
-500
-1000
0
200
400
600
800
Episódio
Sarsa (ε=0.001
Q-learning (ε=

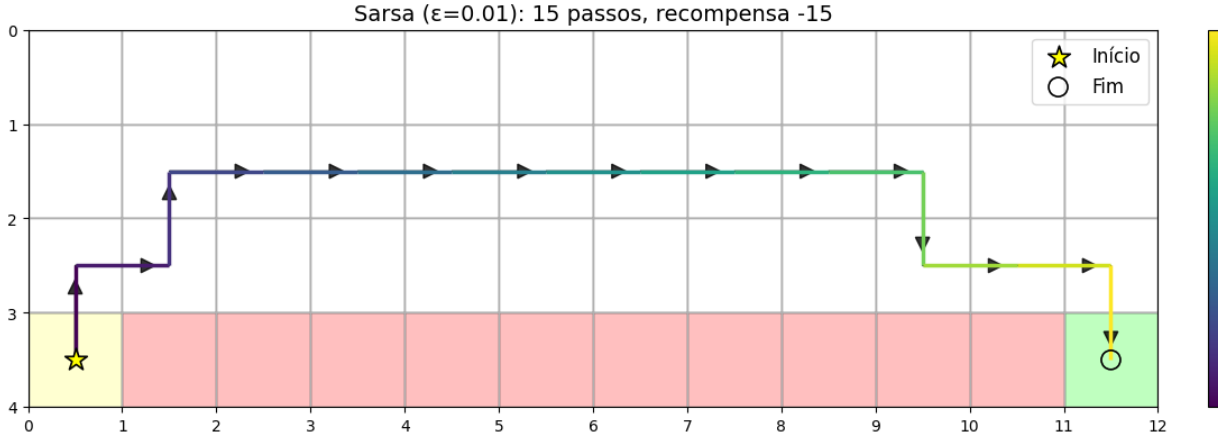

Duração média: Sarsa vs Q-learning (ε = 0.001)
Passos por episódio
(média móvel de 100)
200
100
50
30
20
15
0
200
400
600
800
Episódio
Sarsa (ε=0.001
Q-learning (ε=

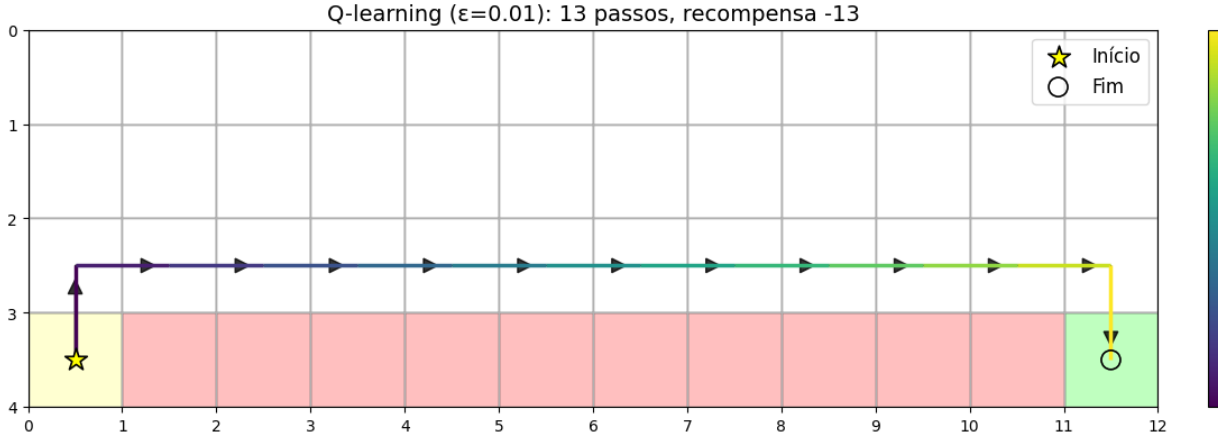

Sarsa (ε=0.001): 13 passos, recompensa -13
Início
Fim
0
1
2
3
4
0
1
2
3
4
5
6
7
8
9
10
11
12

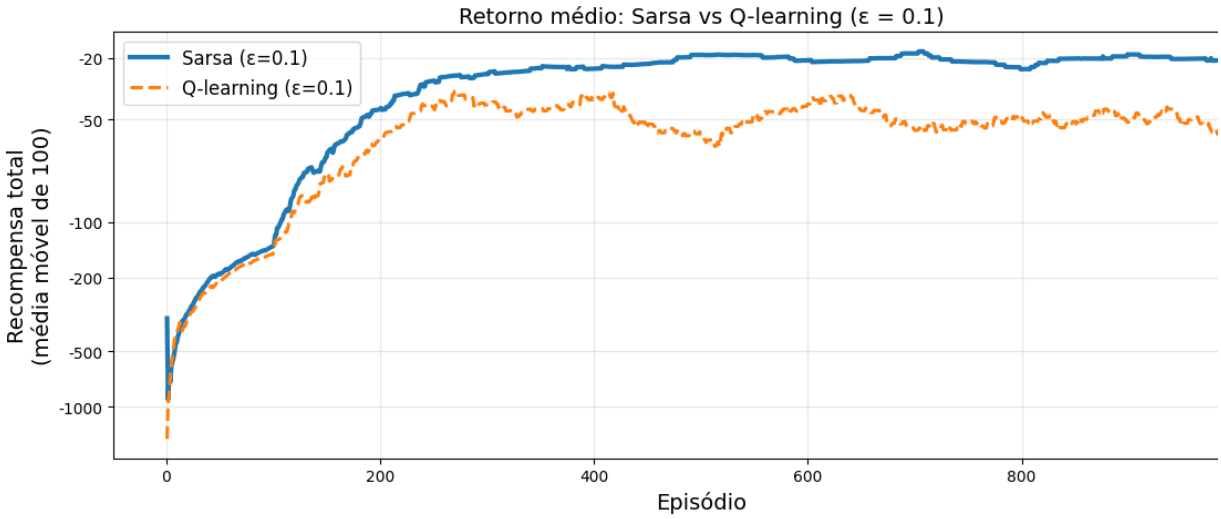

Q-learning (ε=0.001): 13 passos, recompensa -13
Início
Fim
0
1
2
3
4
0
1
2
3
4
5
6
7
8
9
10
11
12

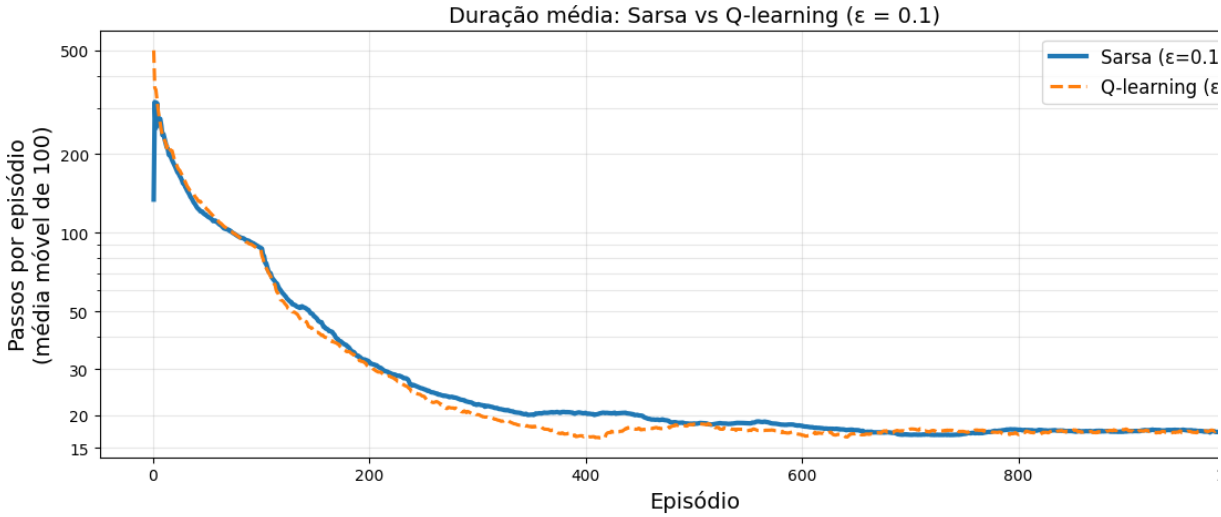

Retorno médio: Sarsa vs Q-learning (ε = 0.01)
Recompensa total
(média móvel de 100)
-20
-50
-100
-200
-500
-1000
0
200
400
600
800
Episódio
Sarsa (ε=0.01
Q-learning (ε=

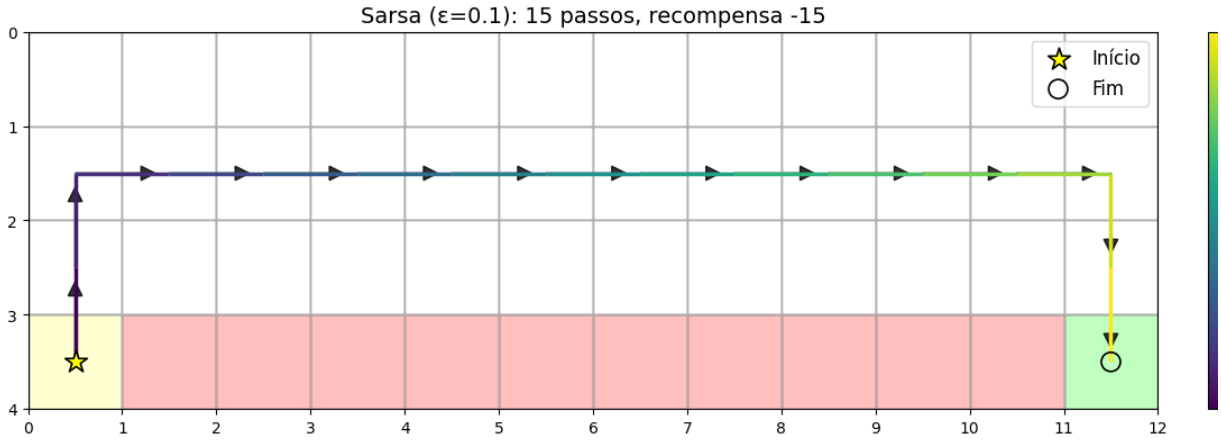

Duração média: Sarsa vs Q-learning (ε = 0.01)
Passos por episódio
(média móvel de 100)
200
100
50
30
20
15
0
200
400
600
800
Episódio
Sarsa (ε=0.01
Q-learning (ε=

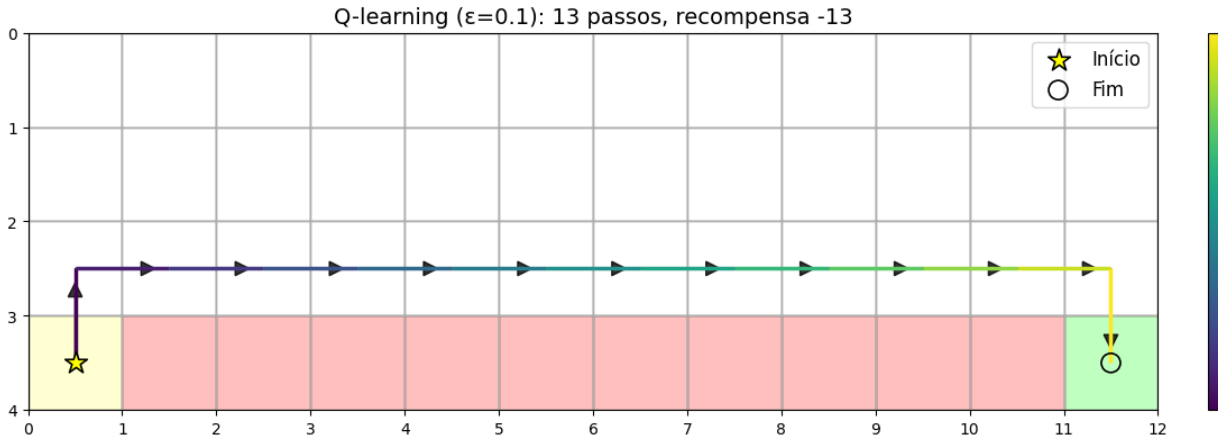

Sarsa (ε=0.01): 15 passos, recompensa -15
Início
Fim
0
1
2
3
4
5
6
7
8
9
10
11
12

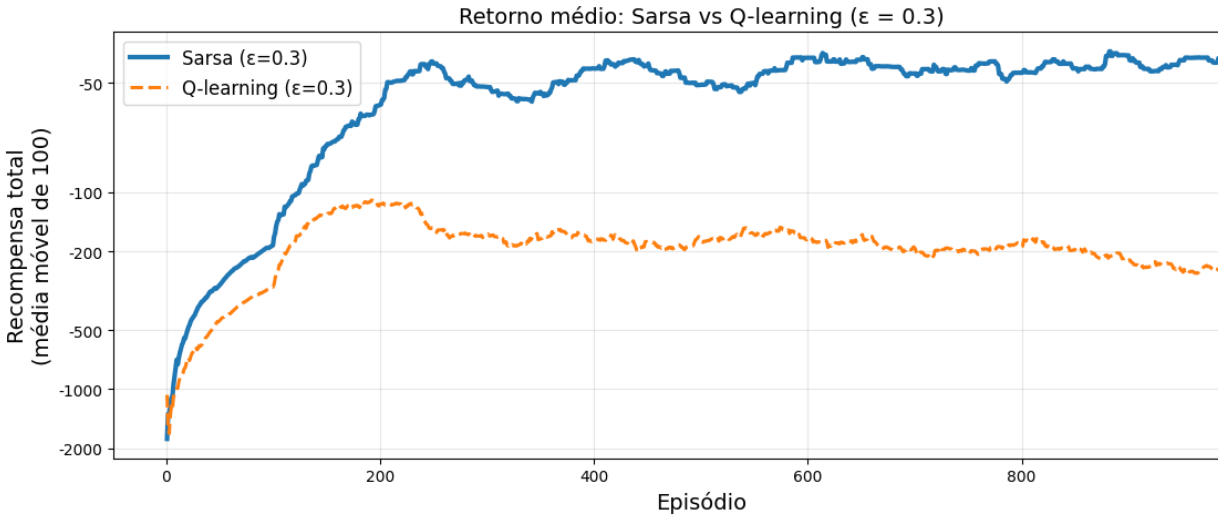

Q-learning (ε=0.01): 13 passos, recompensa -13
Início
Fim
0
1
2
3
4
5
6
7
8
9
10
11
12

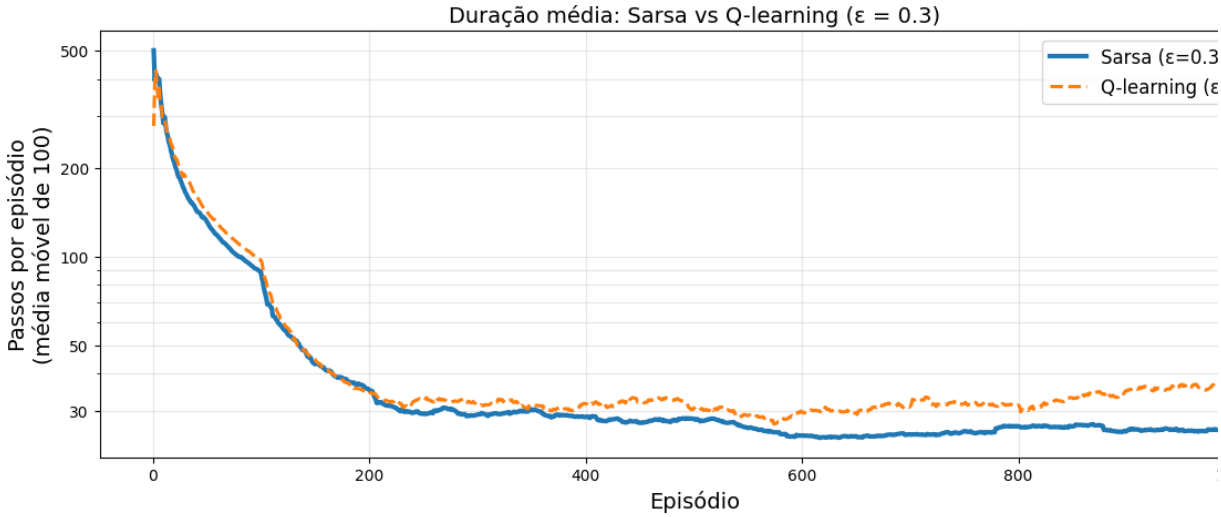

Retorno médio: Sarsa vs Q-learning (ε = 0.1)
Sarsa (ε=0.1)
Q-learning (ε=0.1)
Recompensa total
(média móvel de 100)
-20
-50
-100
-200
-500
-1000
0
200
400
600
800
Episódio

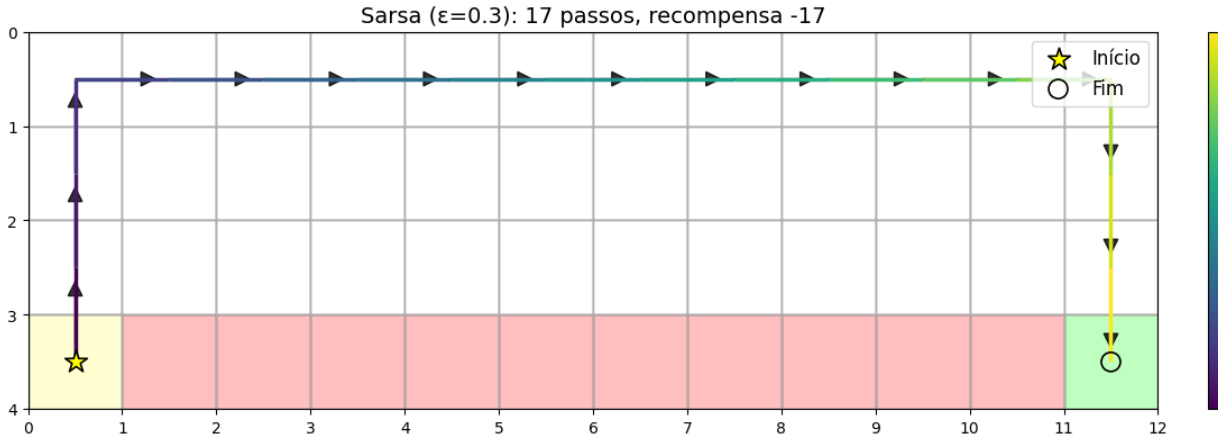

Duração média: Sarsa vs Q-learning (ε = 0.1)
Sarsa (ε=0.1
Q-learning (ε
Passos por episódio
(média móvel de 100)
500
200
100
50
30
20
15
0
200
400
600
800
Episódio

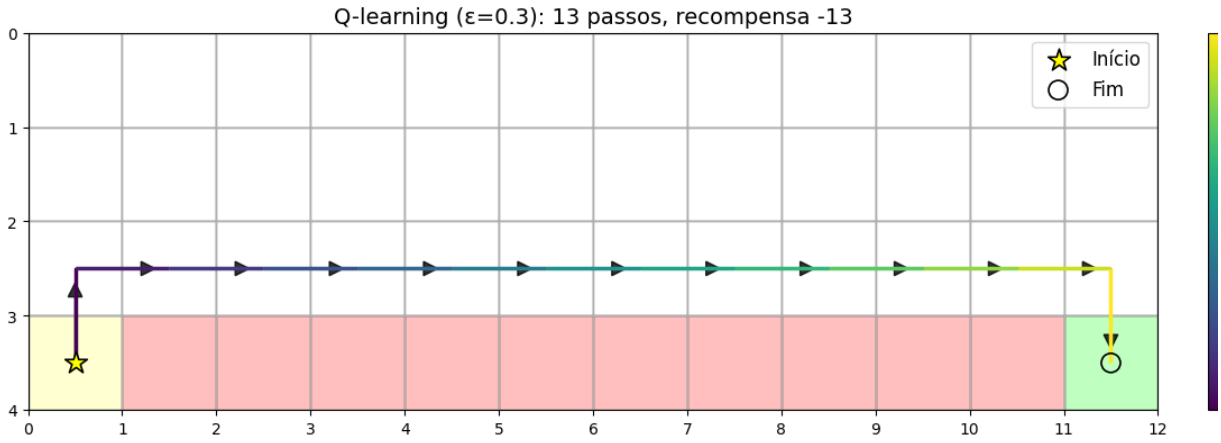

Sarsa (ε=0.1): 15 passos, recompensa -15
Início
Fim
0
1
2
3
4
5
6
7
8
9
10
11
12


Q-learning (ε=0.1): 13 passos, recompensa -13
Início
Fim
0
1
2
3
4
5
6
7
8
9
10
11
12


Retorno médio: Sarsa vs Q-learning (ε = 0.3)
Sarsa (ε=0.3)
Q-learning (ε=0.3)
Recompensa total
(média móvel de 100)
-50
-100
-200
-500
-1000
-2000
0
200
400
600
800
Episódio


Duração média: Sarsa vs Q-learning (ε = 0.3)
Sarsa (ε=0.3
Q-learning (ε
Passos por episódio
(média móvel de 100)
500
200
100
50
30
0
200
400
600
800
Episódio


Sarsa (ε=0.3): 17 passos, recompensa -17
Início
Fim
0
1
2
3
4
0
1
2
3
4
5
6
7
8
9
10
11
12


Q-learning (ε=0.3): 13 passos, recompensa -13
Início
Fim
0
1
2
3
4
0
1
2
3
4
5
6
7
8
9
10
11
12

	Algoritmo	epsilon	Chegada ao alvo (ε-gulosa)	Recompensa média (ε-gulosa)	Duração média (ε-gulosa)	Passos (gulosa)	Recompe (gulo
0	Sarsa	0.001	1.00	-13.00	13.00	13	-1
1	Q-learning	0.001	1.00	-13.00	13.00	13	-1
2	Sarsa	0.010	1.00	-16.70	15.71	15	-1
3	Q-learning	0.010	0.99	-22.21	18.25	13	-1
4	Sarsa	0.100	1.00	-21.33	17.37	15	-1
5	Q-learning	0.100	1.00	-52.45	20.77	13	-1
6	Sarsa	0.300	1.00	-38.33	25.46	17	-1
7	Q-learning	0.300	1.00	-171.31	31.72	13	-1

- **(a)**

Com epsilon = 0.1:

- **Curvas de treino e :** O **Sarsa** atinge maior recompensa total média (significativamente mer negativa, por volta de hBc20 a hBc30, enquanto o Q-learning fica em torno de hBc50 a hBc
- **Trajetória gulosa:** O **Q-learning** tem a trajetória gulosa com maior retorno (hBc13, 13 pass enquanto o Sarsa aprende uma rota mais longa e segura pelo topo da grade (por exemplo, passos, com recompensa hBc15 a hBc17).
- **Explicação pelo alvo TD:** O Sarsa é **on-policy**: seu alvo usa, levando em consideração a a política epsilonhBcgulosa realmente toma. Como ele "sabe" que o agente erra e pode des ele aprende que estados próximos ao penhasco têm valor ruim e prefere contorná-lo por cir (caminho seguro). O Q-learning é **off-policy**: seu alvo usa max_a q, assumindo ações perf futuro. Ele aprende o caminho ótimo estrito, mas sofre muito com quedas durante a explora
- **(b)** À medida que epsilon diminui aproximando-se de 0, o risco de passos exploratórios para abismo desaparece. Com menos exploração, a política seguida pelo Sarsa aproxima-se da puramente gulosa, e o medo de cair no penhasco deixa de existir. Consequentemente, a rot aprendida pelo Sarsa converge exatamente para a rota ótima rente ao penhasco de 13 pass Q-learning, e o desempenho de ambos os algoritmos passa a coincidir.